

Concetti di base

Alfabeto: Insieme finito e non vuoto di simboli

Esempio: $\Sigma = \{0, 1\}$ alfabeto binario

Esempio: $\Sigma = \{a, b, c, \dots, z\}$ insieme di tutte le lettere minuscole

Esempio: Insieme di tutti i caratteri ASCII

Stringa: Sequenza finita di simboli da un alfabeto Σ , es. 0011001
e' una stringa su alfabeto $\{0, 1\}$

Stringa vuota: La stringa con zero occorrenze di simboli da Σ

- La stringa vuota e' denotata con ϵ

Lunghezza di una stringa: Numero di posizioni per i simboli nella stringa.

$|w|$ denota la lunghezza della stringa w

$$|0110| = 4, |\epsilon| = 0$$

Potenze di un alfabeto: Σ^k = insieme delle stringhe di lunghezza k con simboli da Σ

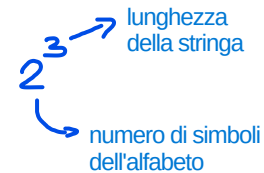
Example: $\Sigma = \{0, 1\}$

$$\Sigma^1 = \{0, 1\}$$

$$\Sigma^2 = \{00, 01, 10, 11\}$$

$$\Sigma^0 = \{\epsilon\}$$

Domanda: Quante stringhe ci sono in Σ^3 ?



L'insieme di tutte le stringhe su Σ e' denotato da Σ^*

$$\Sigma^* = \Sigma^0 \cup \Sigma^1 \cup \Sigma^2 \cup \dots$$

Anche:

$$\Sigma^+ = \Sigma^1 \cup \Sigma^2 \cup \Sigma^3 \cup \dots$$

$$\Sigma^* = \Sigma^+ \cup \{\epsilon\}$$

Concatenazione: Se x e y sono stringhe, allora xy e' la stringa ottenuta collocando una copia di y subito dopo una copia di x

$$x = a_1a_2 \dots a_i, y = b_1b_2 \dots b_j$$

$$xy = a_1a_2 \dots a_ib_1b_2 \dots b_j$$

Esempio: $x = 01101, y = 110, xy = 01101110$

Nota: Per ogni stringa x

$$x\epsilon = \epsilon x = x$$

Equivalenza

Linguaggi:

Se Σ e' un alfabeto, e $L \subseteq \Sigma^*$ allora L e' un linguaggio

Esempi di linguaggi:

- L'insieme delle parole italiane legali
- L'insieme dei programmi C legali
- L'insieme delle stringhe che consistono di n zeri seguiti da n uni

$$\{\epsilon, 01, 0011, 000111, \dots\}$$

- L'insieme delle stringhe con un numero uguale di zeri e di uni

$$\{\epsilon, 01, 10, 0011, 0101, 1001, \dots\}$$

- $L_P =$ insieme dei numeri binari il cui valore e' primo

$$\{\overset{2}{10}, \overset{3}{11}, \overset{5}{101}, \overset{7}{111}, \overset{9}{1011}, \dots\}$$

- Il linguaggio vuoto $\emptyset$
- Il linguaggio $\{\epsilon\}$ consiste della stringa vuota

Nota: $\emptyset \neq \{\epsilon\}$ (Il linguaggio vuoto è diverso dal linguaggio con la sola stringa vuota)

Nota: L'alfabeto Σ e' sempre finito

Automi a stati finiti deterministici

Un DFA e' una quintupla

$$A = (Q, \Sigma, \delta, q_0, F)$$

- Q e' un insieme finito di *stati*
- Σ e' un *alfabeto finito* (= simboli in input)
- δ e' una *funzione di transizione* da $Q \times \Sigma$ a Q ,
cioè: $(q, a) \mapsto p$ (DA prodotto cartesiano di stati e simboli in input A prossimo stato)
- $q_0 \in Q$ e' lo *stato iniziale* (Ha un unico stato iniziale)
- $F \subseteq Q$ e' un insieme di *stati finali*

Esempio: Un automa A che accetta

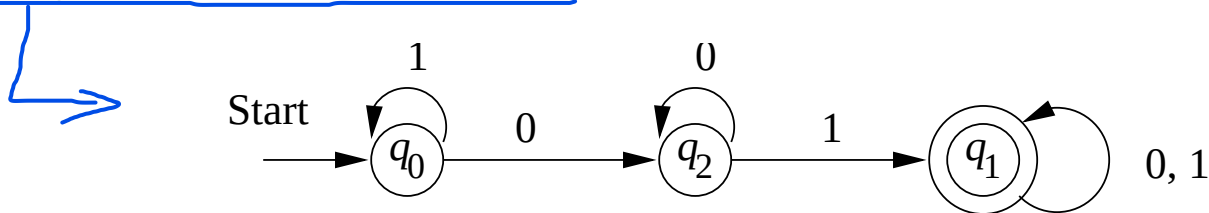
$$L = \{x01y : x, y \in \{0, 1\}^*\}$$

Automa A riconosce le stringhe appartenenti ad L

L'automata $A = (\underbrace{\{q_0, q_1, q_2\}}_{\text{stati}}, \underbrace{\{0, 1\}}_{\text{alfabeto}}, \underbrace{\delta}_{\text{s.i}}, \underbrace{q_0}_{\text{stati finali}}, \underbrace{\{q_1\}}_{\text{stati finali}})$ definito tramite una tabella di transizione:

	0	1
$\rightarrow q_0$	q_2	q_0
$\star q_1$	q_1	q_1
q_2	q_2	q_1

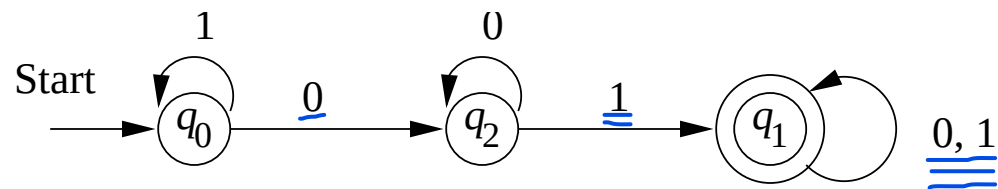
Lo stesso automa A definito, piu' semplicemente, tramite un diagramma di transizione:



Un automa a stati finiti (FA) *accetta* una stringa $w = a_1a_2 \cdots a_n$ se esiste un cammino nel diagramma di transizione che

1. Inizia nello stato iniziale
2. Finisce in uno stato finale (di accettazione)
3. Ha una sequenza di etichette $a_1a_2 \cdots a_n$

Esempio: L'automa a stati finiti



accetta ad esempio la stringa 01101

↪ nell'autociclo dello
stato di accettazione

Per capire dove arrivi con la stringa xa (dove x è una stringa e a l'ultimo carattere):

- Calcola prima dove arrivi con la parte iniziale x (ovvero $\hat{\delta}(q, x)$).
- Dallo stato ottenuto, applica la funzione di transizione singola δ usando l'ultimo carattere a .

In parole povere: È come dare una serie di indicazioni stradali. $\hat{\delta}$ è il risultato finale del viaggio dopo aver seguito tutte le svolte una per una.

La funzione standard $\delta(q, a)$ ti dice dove vai, partendo dallo stato q leggendo il singolo simbolo a . Il delta "col cappello" serve a descrivere il percorso dell'automata su una stringa intera w .

- La funzione di transizione δ può essere estesa a $\hat{\delta}$ che opera su stati e stringhe (invece che su stati e simboli)

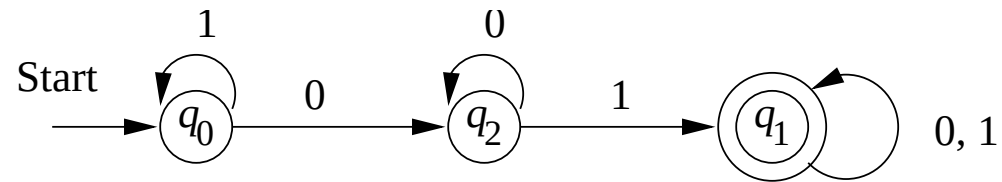
Base: $\hat{\delta}(q, \epsilon) = q$ Se sei nello stato q e non leggi nulla (stringa vuota epsilon), rimani dove sei.

Induzione: $\hat{\delta}(q, xa) = \delta(\hat{\delta}(q, x), a)$ stato in cui sono arrivato

- Formalmente, il *linguaggio accettato da un automa a stati finiti deterministico* A è

$$L(A) = \{w : \hat{\delta}(q_0, w) \in F\}$$

- I linguaggi accettati da automi a stati finiti sono detti *linguaggi regolari*

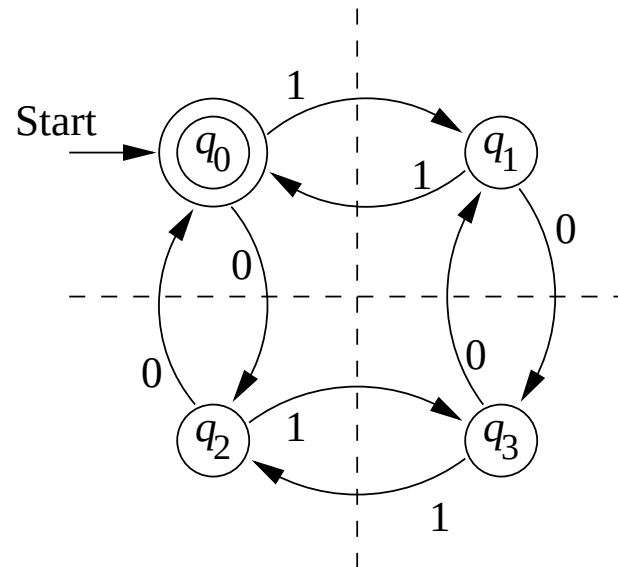


Base: $\hat{\delta}(q, \epsilon) = q$ $\forall q$

Induzione: $\hat{\delta}(q, xa) = \delta(\hat{\delta}(q, x), a)$ $\forall q, x, a$

Esempio: Calcoliamo $\hat{\delta}(q_0, 0110)$

Esempio: DFA che accetta tutte e sole le stringhe con un numero pari di zeri e un numero pari di uni



Rappresentazione tabulare dell'automa

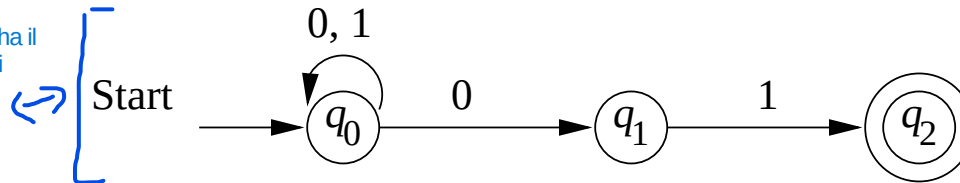
	0	1
* $\rightarrow$ q_0	q_2	q_1
q_1	q_3	q_0
q_2	q_0	q_3
q_3	q_1	q_2

Automi a stati finiti nondeterministici (NFA)

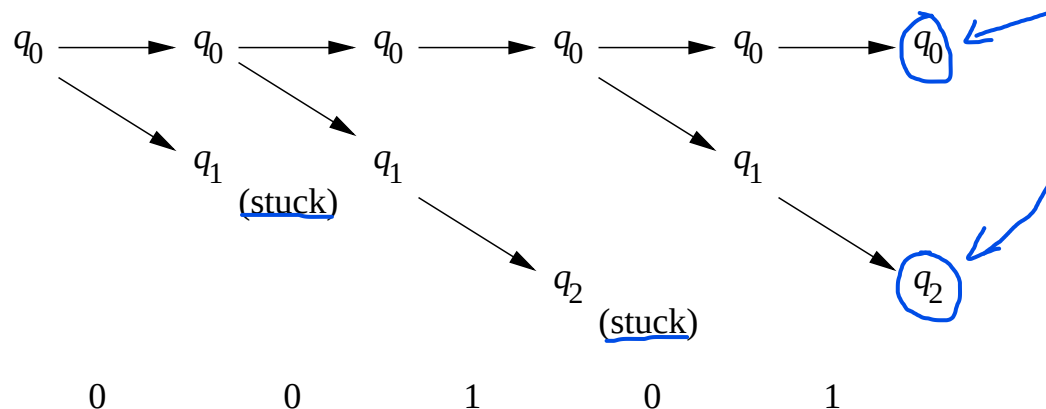
Un NFA accetta una stringa se, tra i tanti possibili ^{cammini}, esiste un cammino che conduce ad uno stato finale.

Esempio: un automa che accetta tutte e solo le stringhe che finiscono in 01.

Perché è non-deterministico questo automa? Si ha il non-determinismo quando da uno stato qualsiasi posso arrivare in più stati con lo stesso simbolo (Esempio: da q_0 posso rimanere in q_0 oppure andare in q_1 se passo all'automata il simbolo 0)



Ecco cosa succede quando l'automata elabora l'input 00101



L'automata accetta la stringa 00101 perché tra i tanti cammini eseguiti, ce ne uno o più che vengono accettati per stato finale di accettazione

Formalmente, un NFA e' una quintupla

$$A = (Q, \Sigma, \delta, q_0, F)$$

- Q e' un insieme finito di stati
- Σ e' un alfabeto finito
- δ e' una funzione di transizione da $Q \times \Sigma$ all'insieme dei sottoinsiemi di Q , cioe': $(q, a) \mapsto \underline{Q'}$ con $\underline{Q' \subseteq Q}$
- $q_0 \in Q$ e' lo *stato iniziale*
- $F \subseteq Q$ e' un insieme di *stati finali*

Questo e' cio' che
cambia dal FA
Deterministico (in
quello di prima, la
funzione di
transizione, data
una coppia
(stato,simbolo),
restituiva un solo
stato di arrivo o
nessuno: ora,
invece, ho un
insieme di stati di
arrivo o nessuno)



Esempio: L' NFA di due pagine fa ^(pagina 12)√e'

$$(\{q_0, q_1, q_2\}, \{0, 1\}, \delta, q_0, \{q_2\})$$

dove δ e' la funzione di transizione

	0	1
$\rightarrow q_0$	$\{q_0, q_1\}$	$\{q_0\}$
q_1	$\emptyset$	$\{q_2\}$
$\star q_2$	$\emptyset$	$\emptyset$

Funzione di transizione estesa $\hat{\delta}$. per NFA

Base:

$$\hat{\delta}(q, \epsilon) = \{q\}$$

In un NFA, la funzione $\delta(q, a)$ non restituisce un singolo stato, ma un insieme di stati (potenzialmente vuoto).

Induzione:

$$\hat{\delta}(q, xa) = \bigcup_{p \in \hat{\delta}(q, x)} \delta(p, a)$$

Induzione: Per capire dove puoi arrivare con la stringa xa :

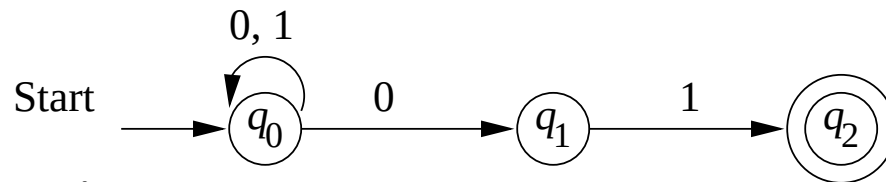
- Guarda l'insieme di tutti gli stati p in cui saresti potuto arrivare leggendo la stringa x .
- Da ognuno di questi stati p , guarda dove puoi andare leggendo l'ultimo carattere a .
- Fai l'unione di tutti questi stati possibili.

Esempio: Calcoliamo $\hat{\delta}(q_0, 0010)$ sulla lavagna

- Formalmente, il *linguaggio accettato da un automa a stati finiti nondeterministico* A è'

$$L(A) = \{w : \hat{\delta}(q_0, w) \cap F \neq \emptyset\}$$

Una stringa w è accettata se esiste almeno un percorso tra quelli possibili che termina in uno stato finale. Non importa se altri percorsi portano a stati non finali o a "vicoli ciechi"; basta che l'intersezione tra gli stati raggiunti ($\hat{\delta}$) e gli stati finali (F) non sia vuota.



Base: $\hat{\delta}(q, \epsilon) = \{q\}$

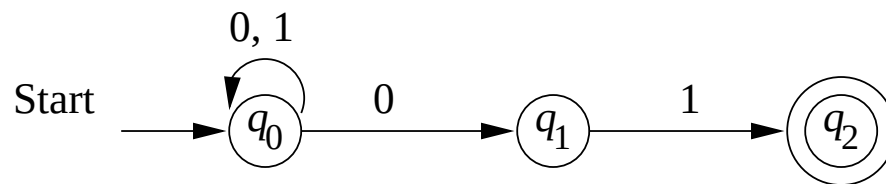
Induzione: $\hat{\delta}(q, xa) = \bigcup_{p \in \hat{\delta}(q, x)} \delta(p, a)$

Esempio: Calcoliamo $\hat{\delta}(q_0, 0010)$

↳ l'NFA è più facile da progettare per l'uomo rispetto ad un DFA

- Gli NFA sono di solito piu' facili da "programmare" dei DFA

Esempio L'NFA



accetta il linguaggio $\{x01 : x \in \Sigma^*\}$.

Equivalenza di DFA e NFA

- Sorprendentemente, per ogni NFA N c'è un DFA D , tale che $L(D) = L(N)$, e viceversa.
- Questo comporta una *costruzione a sottoinsiemi*, un esempio importante di come un automa B può essere costruito da un altro automa A .
- Dato un NFA

$$N = (Q_N, \Sigma, \delta_N, q_0, F_N)$$

costruiremo un DFA

$$D = (Q_D, \Sigma, \delta_D, \{q_0\}, F_D)$$

tali che

$$L(D) = L(N)$$

L'idea Intuitiva: "Simulare il Parallelismo"
Dato che un NFA può trovarsi in un insieme di stati contemporaneamente, l'unico modo per un DFA (che può stare in un solo stato) di imitarlo è quello di avere degli stati che rappresentano "collezioni" di stati dell'NFA. Se l'NFA può trovarsi in {q1,q2}, il DFA si troverà in un singolo stato chiamato [q1,q2].

I dettagli della costruzione a sottoinsiemi:

$Q_D = \{S : S \subseteq Q_N\}$. l'insieme delle Parti di Q_N , cioè l'insieme dei sottoinsiemi di Q_N $\leftrightarrow$ Esempio: Se $Q_N = \{A, B\}$, allora $Q_D = \{\text{emptyset}, \{A\}, \{B\}, \{A, B\}\}$.

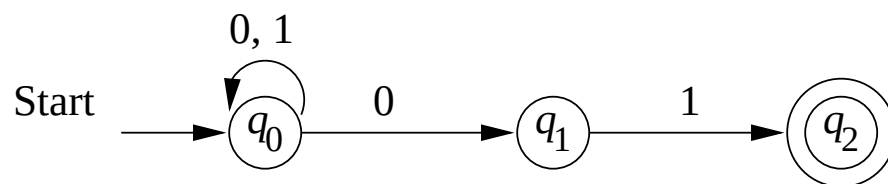
Nota: $|Q_D| = 2^{|Q_N|}$, anche se la maggior parte degli stati in Q_D sono "garbage", cioè non raggiungibili dallo stato iniziale. Spesso si disegnano solo quelli raggiungibili.

- $F_D = \{S \subseteq Q_N : S \cap F_N \neq \emptyset\}$ Uno stato del DFA (che è un insieme di stati dell'NFA) è "finale" se almeno uno degli stati al suo interno era finale nell'NFA.
- Per ogni $S \subseteq Q_N$ e $a \in \Sigma$,

$$\delta_D(S, a) = \bigcup_{p \in S} \delta_N(p, a)$$

Per sapere dove va l'insieme S con il simbolo a, prendi ogni singolo stato p dentro S, guarda dove andrebbe nell'NFA con a, e unisci tutti i risultati in un nuovo super-stato.

Costruiamo δ_D (e F_D) dall' NFA



Per costruire la tabella di transizione:

- Se ho uno stato e devo andare nel successivo, applico le transizioni e vedo dove vado (singolo stato, insieme di stati o insieme vuoto)
- Se ho un insieme di stati e devo andare nel successivo, fisso la transizioni e le applico per ogni stato e vedo dove vado (singolo stato, insieme di stati o insieme vuoto)

Costruiamo δ_D dall' NFA già visto:

	<u>0</u>	1
$\emptyset$	$\emptyset$	$\emptyset$
$\rightarrow \{q_0\}$	$\{\underline{q_0}, \underline{q_1}\}$	$\{q_0\}$
$\{q_1\}$	$\emptyset$	$\{q_2\}$
$\star\{q_2\}$	$\emptyset$	$\emptyset$
$\{\underline{q_0}, \underline{q_1}\}$	$\{\underline{q_0}, \underline{q_1}\}$	$\{q_0, q_2\}$
$\star\{q_0, q_2\}$	$\{q_0, q_1\}$	$\{q_0\}$
$\star\{q_1, q_2\}$	$\emptyset$	$\{q_2\}$
$\star\{q_0, q_1, q_2\}$	$\{q_0, q_1\}$	$\{q_0, q_2\}$

Se in uno stato dell'insieme di stati non ha quella transizione, rimango in quello stato

Essendo che sono insiemi di stati, se da una transizione e da più stati vado nello stesso stato, evito di ripetere lo stesso stato nell'insieme

Nota: Gli stati di D corrispondono a sottoinsiemi di stati di N , ma potevamo denotare gli stati di D in un altro modo, per esempio $A - H$.

	0	1
A	A	A
$\rightarrow B$	E	B
C	A	D
$\star D$	A	A
E	E	F
$\star F$	E	B
$\star G$	A	D
$\star H$	E	F

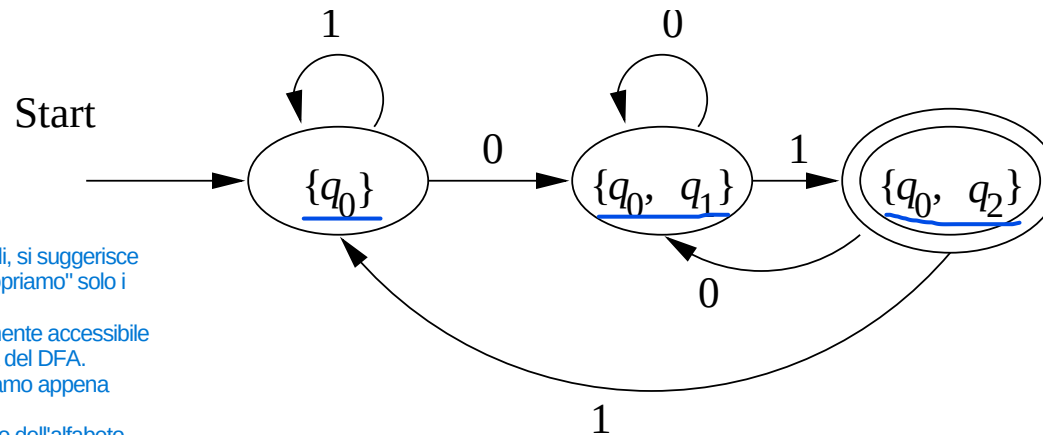
sebbene teoricamente un DFA possa avere 2^n stati, nella maggior parte dei casi reali molti di questi stati non sono raggiungibili partendo dallo stato iniziale. Costruirli tutti sarebbe uno spreco di memoria e tempo.

Possiamo spesso evitare la crescita esponenziale degli stati costruendo la tabella di transizione per D solo per stati accessibili S come segue:

Base: $S = \{q_0\}$ e' accessibile in D

Induzione: Se lo stato S e' accessibile, lo sono anche gli stati $\delta_D(S, a)$ per ogni $a \in \Sigma$.

Esempio: Il "sottoinsieme" DFA solamente con stati accessibili. (Questo DFA ha 3 stati)



Invece di creare prima tutti i 2^n stati e poi collegarli, si suggerisce di procedere al contrario: partiamo dall'inizio e "scopriamo" solo i nuovi stati necessari.

- Base (Punto di partenza): Lo stato $\{q_0\}$ è sicuramente accessibile (è dove iniziamo). Lo mettiamo nella nostra tabella del DFA.

- Induzione (Espansione): Per ogni stato che abbiamo appena scoperto (chiamiamolo S):

> Calcoliamo dove porta con ogni possibile simbolo dell'alfabeto (Σ).

> Se il risultato è un insieme di stati che non abbiamo ancora incontrato, lo aggiungiamo alla tabella come "nuovo stato del DFA".

> Ripetiamo finché non ci sono più nuovi stati da scoprire.

Teorema 2.11: Sia D il DFA ottenuto da un NFA N con la costruzione a sottoinsiemi. Allora $L(D) = L(N)$.

Teorema 2.12: Un linguaggio L e' accettato da un DFA se e solo se L e' accettato da un NFA.

- Il numero di stati del DFA equivalente ad un NFA con n stati e', nel caso peggiore, pari a 2^n stati.

Le epsilon-transizioni permettono all'automa di cambiare stato senza leggere alcun simbolo dall'input.

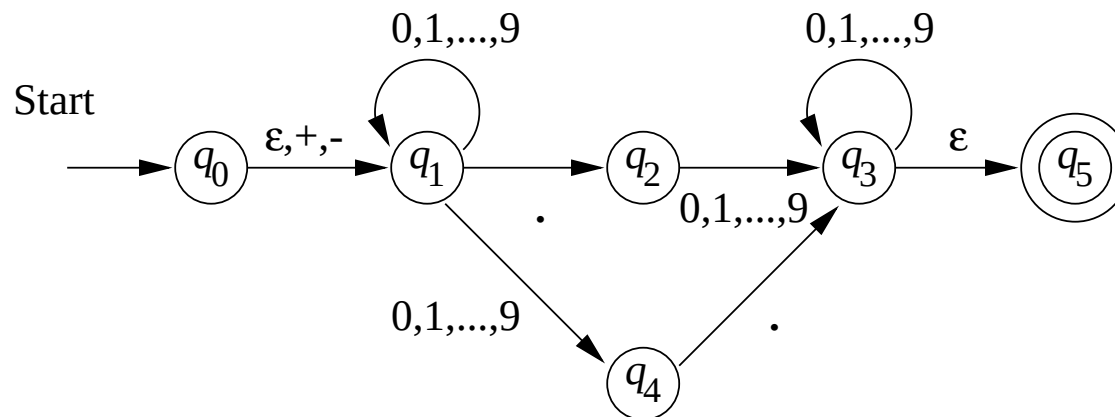
FA con transizioni epsilon

Un ϵ -NFA che accetta numeri decimali consiste di:

1. un segno $+$ o $-$, opzionale
2. una stringa di cifre decimali
3. un punto decimale
4. un'altra stringa di cifre decimali

Una delle stringhe (2) o (4) è opzionale

La regola dice che puoi avere ".5" (manca la parte intera) o "12." (manca la parte frazionaria). L'automa userà i salti vuoti per permettere al flusso di "scavalcare" il blocco della parte intera per andare direttamente al punto decimale, o viceversa.



Un ϵ -NFA e' una quintupla $(Q, \Sigma, \delta, q_0, F)$ dove δ e' una funzione da $Q \times (\Sigma \cup \{\epsilon\})$ all'insieme dei sottoinsiemi di Q .

Esempio: L' ϵ -NFA della pagina precedente

$$E = (\overbrace{\{q_0, q_1, \dots, q_5\}}^{\text{stati}}, \overbrace{\{., +, -, 0, 1, \dots, 9\}}^{\text{alfabeto}}, \delta, q_0, \{q_5\})$$

dove la tabella delle transizioni per δ e'

	ϵ	$+, -$	$.$	$0, \dots, 9$
$\rightarrow q_0$	$\{q_1\}$	$\{q_1\}$	$\emptyset$	$\emptyset$
q_1	$\emptyset$	$\emptyset$	$\{q_2\}$	$\{q_1, q_4\}$
q_2	$\emptyset$	$\emptyset$	$\emptyset$	$\{q_3\}$
q_3	$\{q_5\}$	$\emptyset$	$\emptyset$	$\{q_3\}$
q_4	$\emptyset$	$\emptyset$	$\{q_3\}$	$\emptyset$
$\star q_5$	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$

Epsilon-chiusura

Calcoliamo l'ECLOSURE di

Chiudiamo uno stato aggiungendo tutti gli stati raggiungibili da lui tramite una sequenza $\epsilon\epsilon\cdots\epsilon$
 di transizioni epsilon

Definizione induttiva di $ECLOSE(q)$

Base:

$q \in ECLOSE(q)$

Induzione:

$p \in ECLOSE(q)$ and $r \in \delta(p, \epsilon) \Rightarrow$
 $r \in ECLOSE(q)$

La definizione serve a catturare l'idea che, se sei in uno stato q , sei automaticamente anche in tutti gli stati che puoi raggiungere senza consumare alcun carattere della stringa di input.

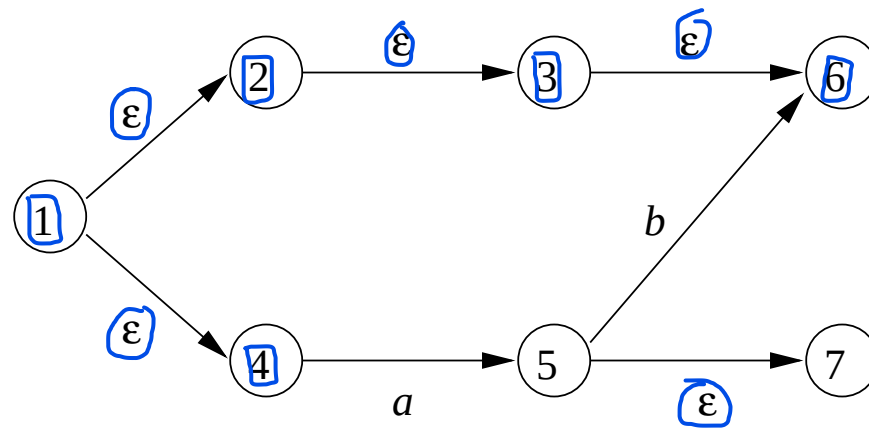
- Base: Questo è il principio di riflessività. Ogni stato può raggiungere se stesso leggendo "nulla" (epsilon). Quindi, il punto di partenza è sempre incluso.

- Induzione: Questo definisce la transitività. Se sei già arrivato in uno stato p "gratuitamente" partendo da q , e da p esiste una freccia epsilon che porta a r , allora anche r fa parte della famiglia di stati raggiungibili gratuitamente da q .

In sintesi: $ECLOSE(q)$ è l'insieme di tutti gli stati raggiungibili da q seguendo zero, una o più transizioni epsilon.

Poiché $\delta(p, \epsilon)$ restituisce un insieme di stati, usiamo il simbolo di appartenenza. Potrebbero esserci molti stati raggiungibili da p tramite epsilon (esempio: $\delta(p, \epsilon) = \{r, s, t\}$). Dire che r vi appartiene significa che r è una delle possibili destinazioni immediate.

Esempio di ϵ -chiusura



Per esempio,

$$\text{ECLOSE}(1) = \{1, 2, 3, 4, 6\}$$

Non c'è il 7 perché non è raggiungibile da 1 con una sequenza di transizioni epsilon

- Definizione induttiva di $\hat{\delta}$ per automi ϵ -NFA

Base:

$$\hat{\delta}(q, \epsilon) = \text{ECLOSE}(q)$$

In un epsilon-NFA, anche se non leggi alcun carattere (epsilon), l'automa non è confinato nello stato q . Può "espandersi" in tutti gli stati raggiungibili gratuitamente. Quindi, Se l'automa parte in q e riceve una stringa vuota, la sua posizione finale è l'intera "bolla" di stati raggiungibili via epsilon.

Induzione:

$$\hat{\delta}(q, xa) = \bigcup_{p \in \hat{\delta}(q, x)} \left(\bigcup_{t \in \delta(p, a)} \text{ECLOSE}(t) \right)$$

Identifica l'insieme di stati p in cui l'automa si trova dopo aver letto la stringa precedente x .

Per ogni stato p raggiunto, l'automa legge il simbolo reale a e si sposta in un nuovo insieme di stati t .

Quindi, non bisogna prendere solo lo stato t che raggiungo con $\delta(p, a)$, ma anche gli stati che da t si arriva con successive transizioni epsilon.

Una volta arrivato in t tramite il simbolo a , l'automa attiva immediatamente tutte le possibili transizioni epsilon successive.

- Linguaggio L accettato è ancora definito $\{w : \hat{\delta}(q_0, w) \cap F \neq \emptyset\}$

Questa parte è identica a quella degli NFA. Una stringa w fa parte del linguaggio se, partendo dallo stato iniziale q_0 ed applicando la funzione estesa $\hat{\delta}$, l'insieme di stati risultante contiene almeno uno stato che appartiene all'insieme degli stati finali F .

Descrizione formale di come gli epsilon-NFA possano essere convertiti in DFA.

Equivalenza di DFA e ϵ -NFA

Dato un ϵ -NFA

$$E = (Q_E, \Sigma, \delta_E, q_0, F_E)$$

costruiremo un DFA

$$D = (Q_D, \Sigma, \delta_D, q_D, F_D)$$

tale che

$$L(D) = L(E)$$

Dettagli della costruzione:

- $Q_D =$

$$\{S : S \subseteq Q_E \text{ e } S = \bigcup_{s \in S} \text{ECLOSE}(s)\}$$

Ogni stato S del DFA è un sottoinsieme di stati dell'NFA originale che deve essere già chiuso rispetto a epsilon ($S = \bigcup \{s \text{ in } S\} \text{ECLOSE}(s)$).

Stato iniziale del DFA

- $q_D = \text{ECLOSE}(q_0)$

→ stato iniziale dell'epsilon-NFA

Nel DFA, lo stato di partenza non è semplicemente q_0 , ma la sua chiusura: $q_D = \text{ECLOSE}(q_0)$. Questo perché l'automa è "già" in tutti quegli stati prima ancora di iniziare.

- $F_D = \{S : S \in Q_D \text{ e } S \cap F_E \neq \emptyset\}$

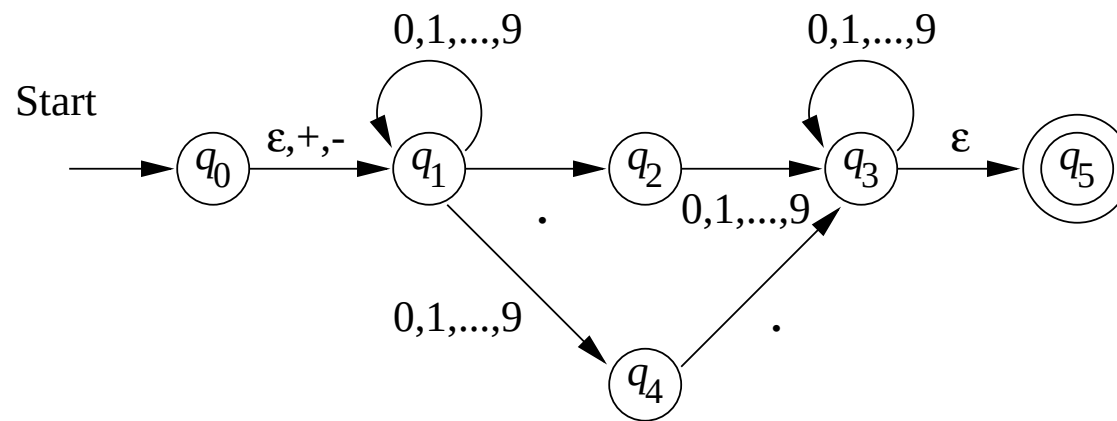
→ Un set S è uno stato finale nel DFA se contiene almeno uno stato che era finale nell'NFA

- $\delta_D(S, a) =$

$$\bigcup_{t \in S} \left(\bigcup_{p \in \delta_E(t, a)} \text{ECLOSE}(p) \right)$$

Per muoversi da un super-stato S con il simbolo a , il DFA guarda dove portano tutti gli stati t in S con quel simbolo e calcola la ECLOSURE di ogni risultato

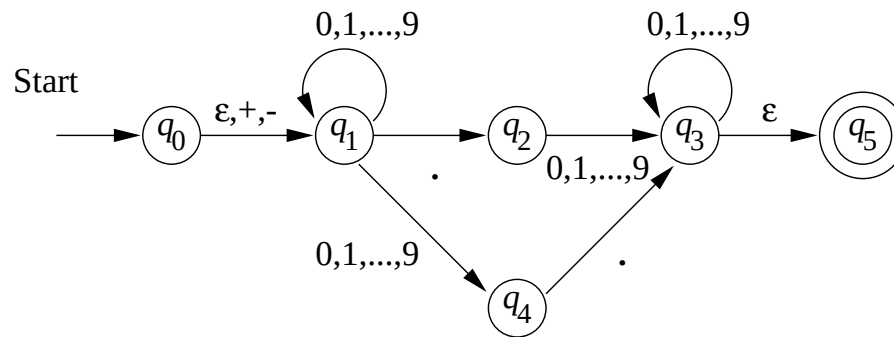
Esempio: ϵ -NFA E



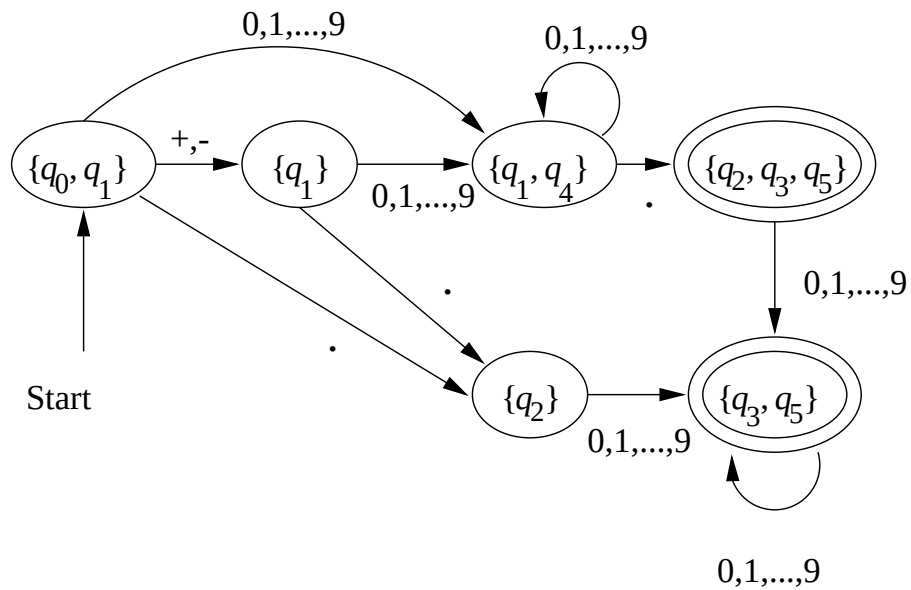
DFA D corrispondente ad E



Esempio: ϵ -NFA E



DFA D corrispondente ad E



Teorema 2.22: Un linguaggio L e' accettato da un ϵ -NFA E se e solo se L e' accettato da un DFA.

Espressioni regolari

Un FA (NFA o DFA) e' un metodo per costruire una macchina che riconosce linguaggi regolari.

Una *espressione regolare* e' un modo dichiarativo per descrivere un linguaggio regolare.

Esempio: $01^* + 10^*$

Le espressioni regolari sono usate, ad esempio, in

1. comandi UNIX (grep)
2. strumenti per l'analisi lessicale di linguaggi: es. Lex (Lexical analyzer generator), Flex (Fast Lex) e ANTLR (Another Tool For Language Recognition).

Operazioni sui linguaggi

Unione:

$$L \cup M = \{w : w \in L \text{ o } w \in M\}$$

Concatenazione:

$$L.M = \{w : w = xy, x \in L, y \in M\}$$

Potenze:

$$L^0 = \{\epsilon\}, L^1 = L, L^{k+1} = L.L^k$$

Rappresentano la concatenazione di L con se stesso k volte.

Esempio: $L^3 = L^0 L^1 L^2$

Chiusura di Kleene:

$$L^* = \bigcup_{i \geq 0} L^i$$

Rappresenta l'unione di tutte le potenze di L da zero ad infinito ($L^0 \cup L^1 \cup L^2 \dots$). Rappresenta il concetto di "zero o più ripetizioni".

La chiusura di Kleene è l'unione di tutte le potenze:
 $\text{emptyset}^* = \text{emptyset}^0 \cup \text{emptyset}^1 \cup \text{emptyset}^2 \dots$

Sostituendo i valori trovati sopra: $\text{emptyset}^* = \{\epsilon\} \cup \text{emptyset} \cup \text{emptyset} \dots = \{\epsilon\}$

Domanda: Cosa sono $\emptyset^0$, $\emptyset^i$, e $\emptyset^*$?

Per definizione universale delle potenze di un linguaggio, $L^0 = \{\epsilon\}$ per qualsiasi linguaggio L. Anche se partiamo dal nulla, la "zero-esima" potenza produce sempre l'insieme contenente la sola stringa vuota.

Se provi a concatenare l'insieme vuoto con se stesso anche solo una volta ($i=1, 2, \dots$), il risultato rimane l'insieme vuoto perché la concatenazione richiede di scegliere una stringa da L e una da M. Se uno dei due è vuoto, non puoi scegliere nulla, quindi non puoi formare nessuna stringa.

Leggi algebriche per i linguaggi

- $L \cup M = M \cup L$.

L'unione e' *commutativa*.

- $(L \cup M) \cup N = L \cup (M \cup N)$.

L'unione e' *associativa*.

- $(LM)N = L(MN)$.

La concatenazione e' *associativa*

Nota: La concatenazione non e' commutativa, *cioe'*, esistono L e M tali che $LM \neq ML$.

- $\emptyset \cup L = L \cup \emptyset = L.$

$\emptyset$ e' l'*identita'* per l'unione.

- $\{\epsilon\}L = L\{\epsilon\} = L.$

$\{\epsilon\}$ e' l'*identita'* *sinistra* e *destra* per la concatenazione.

- $\emptyset L = L\emptyset = \emptyset.$

$\emptyset$ e' l'*annichilatore* *sinistro* e *destro* per la concatenazione.

Per formare una stringa nel linguaggio concatenato $L.M$, devi poter scegliere una stringa da L e una da M . Se uno dei due insiemi è vuoto (emptyset), è impossibile effettuare la scelta. Di conseguenza, non è possibile generare alcuna stringa.

- $L(M \cup N) = LM \cup LN.$

La concatenazione e' *distributiva a sinistra* sull'unione.

- $(M \cup N)L = ML \cup NL.$

La concatenazione e' *distributiva a destra* sull'unione.

- $L \cup L = L.$

L'unione e' *idempotente*.

- $\emptyset^* = \{\epsilon\}, \quad \{\epsilon\}^* = \{\epsilon\}.$

- $L^+ = LL^* = L^*L, \quad L^* = L^+ \cup \{\epsilon\}$
- $(L^*)^* = L^*$

$= L^1 L^2 L^3 \dots$

$L^0 = \{\epsilon\}$

Costruire le espressioni regolari

Definizione induttiva di espressioni regolari:

Base: ϵ e $\emptyset$ sono espressioni regolari
 $L(\epsilon) = \{\epsilon\}$, e $L(\emptyset) = \emptyset$.

Se $a \in \Sigma$, allora a e' un'espressione regolare
 $L(a) = \{a\}$.

Induzione:

Se E e' un'espr. regolare, allora (E) e' un'espr. regolare
 $L((E)) = L(E)$.

Se E e F sono espr. regolari, allora $E + F$ e' un'espr. regolare
 $L(E + F) = L(E) \cup L(F)$.

Se E e F sono espr. regolari, allora $E.F$ e' un'espr. regolare
 $L(E.F) = L(E).L(F)$.

Se E e' un'espr. regolare, allora E^* e' un'espr. regolare
 $L(E^*) = (L(E))^*$.

Esempio:

$E = (0+1).(1^*)$

$L(E) = ?$: Quali sono le stringhe che si costruiscono con l'espressione regolare E?

$L(0) = 0$

$L(1) = 1$

$L(0+1) = L(0) \cup L(1) = \{0,1\}$

$L(1^*) = (L(1))^* = \{1\}^* = \text{Unione}_{i \geq 0} \{1\}^i = \{\epsilon, 1, 11, \dots\}$

$L(E) = \{0,1\}.\{1^i \text{ t.c } i \geq 0\} = \{01^i, 11^i \text{ t.c } i \geq 0\} = \{a1^i \text{ t.c } a \text{ appartiene a } \{0,1\} \text{ AND } i \geq 0\}$

Esempio: Espressioni regolari per

$L = \{w \in \{0,1\}^* : 0 \text{ e } 1 \text{ alternati in } w\}$

L'espressione fornita scompone il problema in quattro casi base basati su come la stringa inizia e finisce:

$(01)^*$: Inizia con 0, finisce con 1 (lunghezza pari o zero).

$(10)^*$: Inizia con 1, finisce con 0 (lunghezza pari o zero).

$0(10)^*$: Inizia con 0, finisce con 0 (lunghezza dispari).

$1(01)^*$: Inizia con 1, finisce con 1 (lunghezza dispari).

L'operatore "+" agisce come un OR logico (unione), indicando che una stringa valida può appartenere a uno qualsiasi di questi quattro casi.

$(01)^* + (10)^* + 0(10)^* + 1(01)^*$

o, equivalentemente,

$(\epsilon + 1)(01)^*(\epsilon + 0)$

Qui usiamo la concatenazione per gestire i bordi della stringa:

- $(\epsilon + 1)$: La stringa può iniziare opzionalmente con un 1.

- $(01)^*$: Il "cuore" della stringa, una sequenza ripetuta di 01.

- $(\epsilon + 0)$: La stringa può terminare opzionalmente con uno 0.

Ordine di precedenza per gli operatori:

1. **Chiusura** (Stella di Kleene) (stessa precedenza della potenza)
2. **Concatenazione (punto)** (stessa precedenza della moltiplicazione)
3. **Piu' (+)**

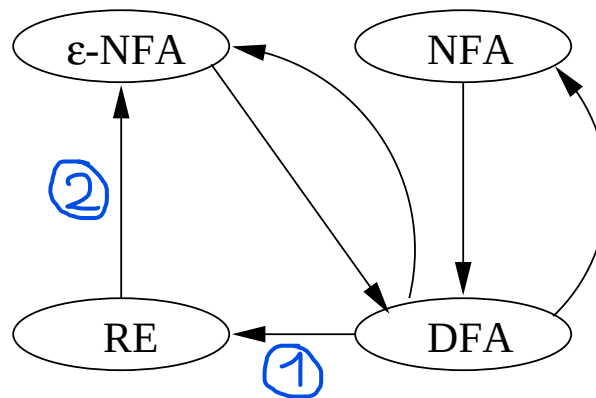
Perché è equivalente? Se scegliamo epsilon a sinistra e 0 a destra, otteniamo $(01)^*0$, che copre le stringhe che iniziano con 0 e finiscono con 0. Se scegliamo 1 a sinistra e 0 a destra, otteniamo $1(01)^*0$, che è un altro modo di scrivere $(10)^*10$ o semplicemente una sequenza che inizia con 1 e finisce con 0.

Esempio: $01^* + 1$ e' raggruppato in $(0(1)^*) + 1$

in alcuni testi l'unione viene indicata con il simbolo | (pipe) invece del +. Il principio rimane identico: è l'operatore più "debole" della gerarchia.

Equivalenza di FA e espr. regolari

Abbiamo già mostrato che DFA, NFA, e ϵ -NFA sono tutti equivalenti.



Per mostrare che gli FA sono equivalenti alle espressioni regolari dobbiamo stabilire che

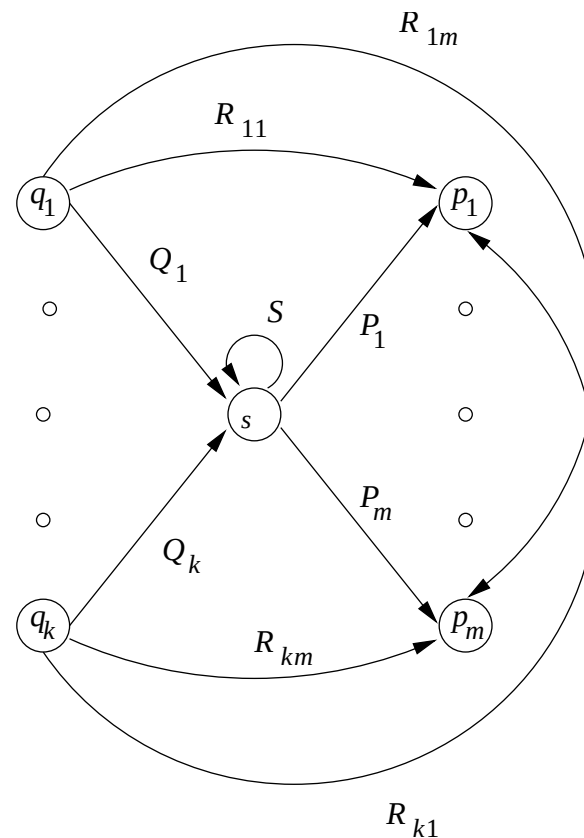
1. Per ogni DFA A possiamo trovare (costruire, in questo caso) un'espressione regolare R , tale che $L(R) = L(A)$.
2. Per ogni espressione regolare R esiste un ϵ -NFA A , tale che $L(A) = L(R)$.

1

Da DFA a espressioni regolari

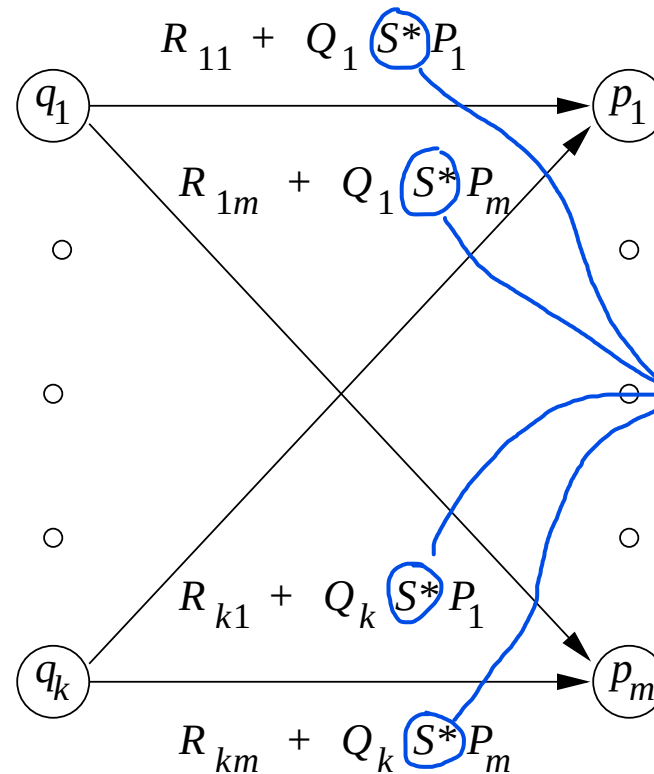
Teorema 3.4: Per ogni DFA $A = (Q, \Sigma, \delta, q_0, F)$ esiste una espressione regolare R , tale che $L(R) = L(A)$.

Trasformiamo l'automa etichettando gli archi con espressioni regolari di simboli



① Elimino tutti gli stati che non sono né stati iniziali né stati di accettazione

Ora eliminiamo lo stato s .



per fare ciò, per ogni stato selezionato non di accettazione né iniziale, scrivo i suoi predecessori (stati da cui parte la transizione ed arriva in esso) e successori (transizioni che fa partire lo stato selezionato verso il successore). Poi creo una tabella dove faccio, tutte le combinazioni predec-succ, dove per ogni combinazione ho una espressione regolare: "transizione predec-succ diretta" + "transizione predec-succ passante per lo stato da eliminare"

Se ho un autociclo su stato da eliminare faccio: $stato^{**}$ (però se ho autociclo nella transizione predec-succ diretta, dove predec e succ è lo stesso stato, l'autociclo viene eseguito una sola volta quindi non ha la Stella di Kleene)

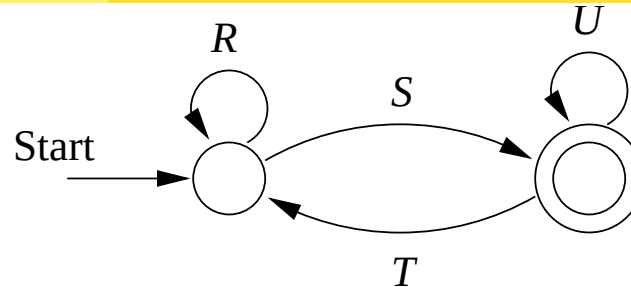
Per lo stato accettante q eliminiamo dall'automa originale tutti gli stati eccetto q_0 e q .

②

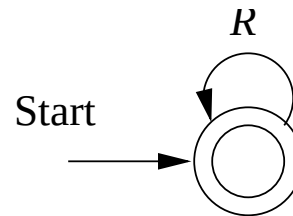
Mi fisso su ogni stato di accettazione ed elimino gli altri tranne lo stato iniziale (può capitare che lo stato di accettazione sia lo stato iniziale): quindi creo per ogni visione su uno stato di accettazione fisso, una espressione regolare

Se lo stato iniziale é stato di accettazione, quando fisso un altro stato di accettazione, quello stato rimane solo come stato iniziale e non anche come di accettazione

② Per ogni $q \in F$ saremo rimasti con A_q della forma



e che corrisponde all'espressione regolare $E_q = (R + SU^*T)^*SU^*$
o con A_q della forma



che corrisponde all'espressione regolare $E_q = R^*$

- L'espressione finale é'

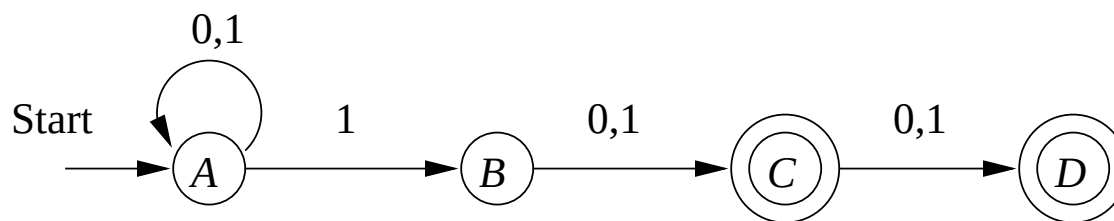
$$\bigoplus_{q \in F} E_q$$

Dopo aver fissato tutti gli stati di accettazione e creato per ognuno l'espressione regolare, le sommo

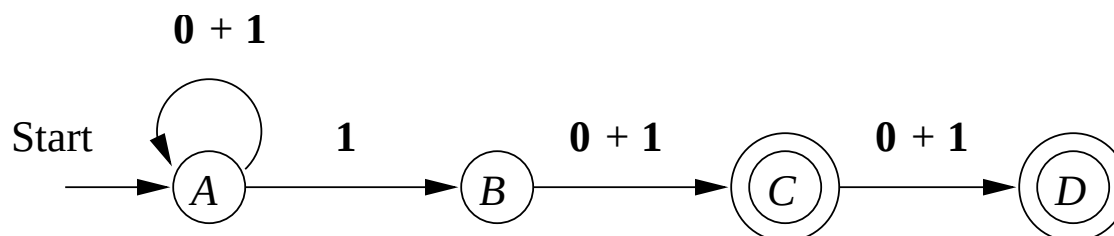
In realta' l'algoritmo e' applicabile anche a NFA!

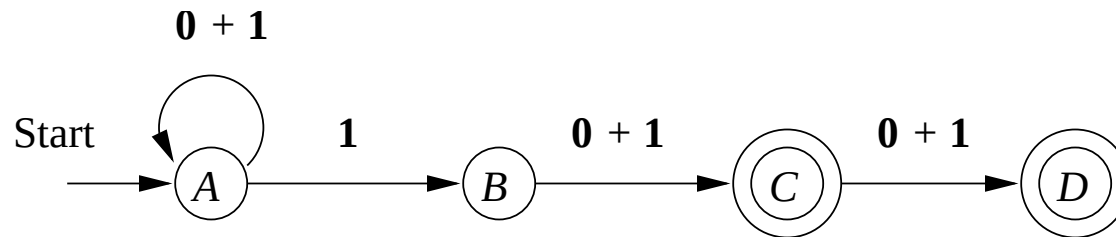
Esempio: $\mathcal{A}$, dove

$$L(\mathcal{A}) = \{w : w = x1b \text{ o } w = x1bc, \ x \in \{0,1\}^*, \{b,c\} \subseteq \{0,1\}\}$$



La trasformiamo in un automa con espressioni regolari come etichette

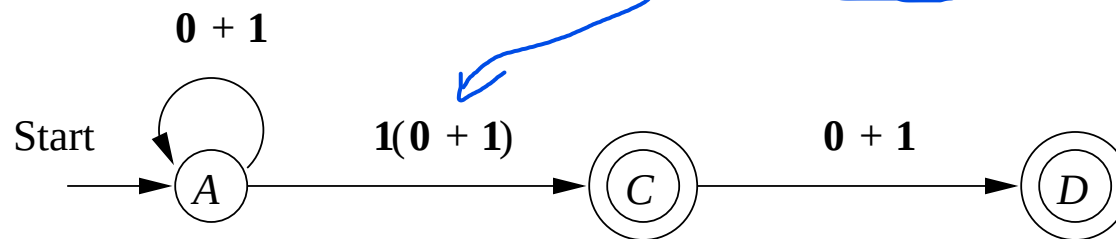




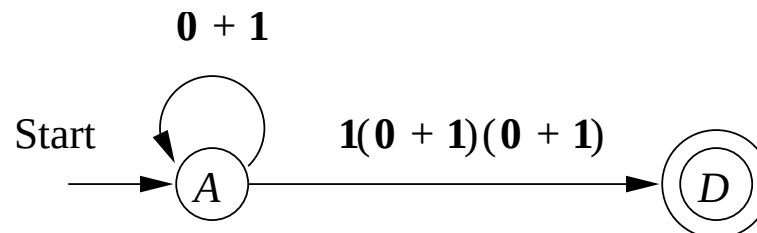
① Eliminiamo lo stato B

$$A \rightarrow C = \emptyset + 1\emptyset(0+1)$$

Questo perché A è predecessore di B e C è successore di B

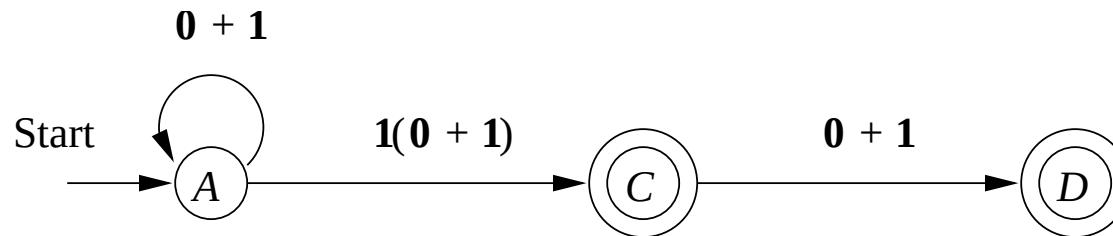


② Fisso D come unico stato di accettazione
Poi eliminiamo lo stato C e otteniamo $\mathcal{A}_D$

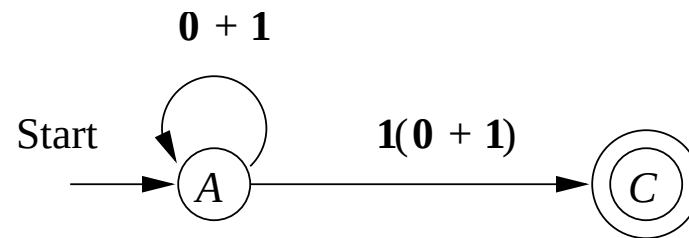


con espressione regolare $(0+1)^*1(0+1)(0+1)$

Da



2 → Fisso C come unico stato di accettazione
possiamo eliminare D e ottenere $\mathcal{A}_C$



con espressione regolare $(0 + 1)^*1(0 + 1)$

- L'espressione finale e' la somma delle due espressioni regolari precedenti:

$$(0 + 1)^*1(0 + 1)(0 + 1) + (0 + 1)^*1(0 + 1)$$

2

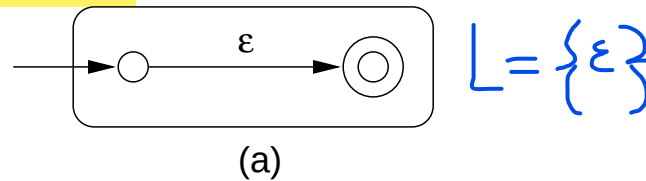
Da espressioni regolari a ϵ -NFA

Teorema 3.7: Per ogni espressione regolare R possiamo costruire un ϵ -NFA A (con un unico stato finale, diverso da quello iniziale), tale che $L(A) = L(R)$.

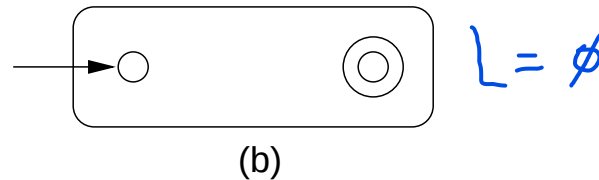
Prova: Per induzione strutturale:

Base: Automa per ϵ , $\emptyset$, e a .

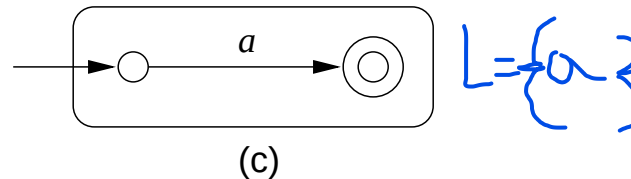
Linguaggio dell'Automa (a)



Linguaggio dell'Automa (b)



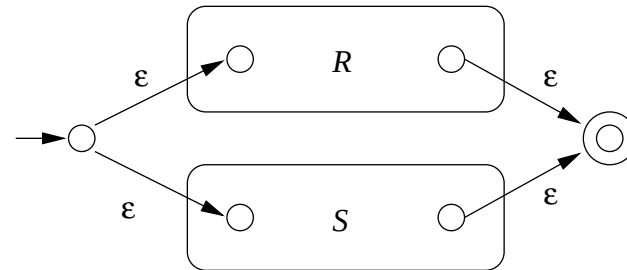
Linguaggio dell'Automa (c)



Invariante per ogni automa durante Algoritmo: un unico stato di accettazione e lo stato iniziale non è mai di accettazione

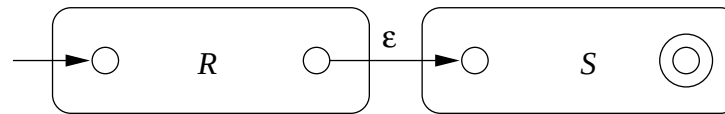
Invariante per ogni automa durante
Algoritmo: un unico stato di accettazione e
lo stato iniziale non é mai di accettazione

Induzione: Automa per $R + S$, RS , e R^*



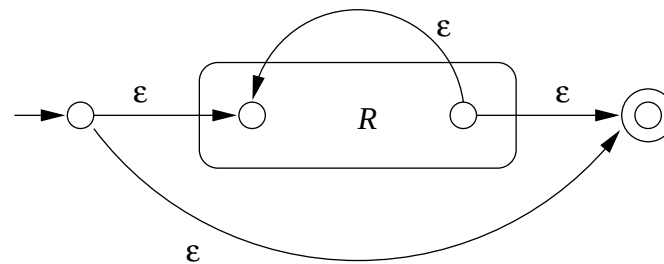
(a)

Linguaggio dell'automa (a): unione di tutte le stringhe riconosciute da automi R e S



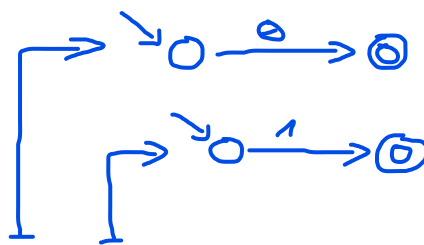
(b)

Linguaggio dell'automa (b): concatenazione di tutte le stringhe riconosciute da automi R e S



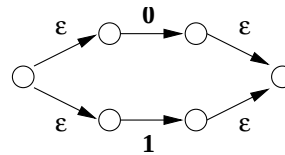
(c)

Linguaggio dell'automa (c): riconosciute tutte le stringhe concatenate 0 o più volte, riconosciute da R



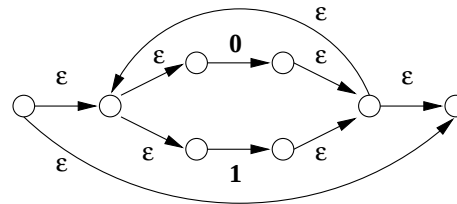
Esempio: Trasformiamo $(0 + 1)^*1(0 + 1)$

(a): $0+1$



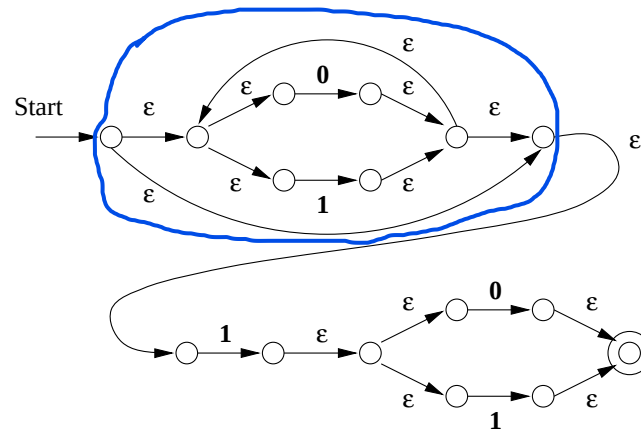
(a)

(b): $(0+1)^*$



(b)

(c): $((0+1)^*1(0+1))$



(c)

Proprieta' dei Linguaggi regolari

↳ Usato per dimostrare che un linguaggio fa parte dei regolari o no

- *Pumping Lemma*. Ogni linguaggio regolare soddisfa il pumping lemma. Dato un linguaggio, se usando pumping lemma si ottiene una contraddizione allora esso non e' regolare.
- *Proprieta' di chiusura*. Come costruire automi da componenti usando delle operazioni e per quali operazioni è possibile farlo. Ad esempio dati L e M possiamo costruire un automa per $L \cap M$.
- *Proprieta' di decisione*. Analisi computazionale di automi, ad esempio quando due automi sono equivalenti.
- *Tecniche di minimizzazione*. Possiamo risparmiare costruendo automi piu' piccoli.

Il Pumping Lemma, idea di base

Supponiamo, per assurdo che $L = \{0^h 1^h : h \geq 0\}$ sia regolare.

Allora deve essere accettato da un qualche DFA A . Chiamo n il numero degli stati di A .

é maggiore uguale perché se il numero di stati é n , il numero di transizioni sono $n-1$

Se A riceve in input una stringa di lunghezza $\geq n$ allora lungo il cammino di riconoscimento incontra due volte uno stesso stato.

Es. supponiamo che A legga 0^n . Considero gli stati che incontra:

ϵ	p_0 ($= q_0$ stato iniziale di A)
0	p_1
00	p_2
$\dots$	$\dots$
0^n	p_n

Sono $n + 1$

$\Rightarrow \exists i < j : p_i = p_j$, chiamiamo q questo stato

(dopo aver letto 0^i , legge altri 0 fino a 0^j , facendo **un ciclo** su q)

la stringa toccherà due volte lo stesso stato (quindi fa un ciclo)

non riconosce la differenza di
quantità tra gli 'i' zeri con i 'j'
zeri, quindi non so nemmeno
quanti zeri sono entrati
nell'automa (se entro nel ciclo
quando la stringa $\geq n$)

A ha dei **problemi di memoria!**

Va in q sia ricevendo in input 0^i che ricevendo in input 0^j !!

Siccome abbiamo supposto che A accetti $\{0^h 1^h : h \geq 0\}$ si avrà:

- l'automa, dovendo accettare $0^i 1^i$, sarà tale che $\hat{\delta}(q, 1^i) \in F$
- ma allora anche $0^j 1^i$ sarà accettata!! (**assurdo!**)

cioè dallo stato q con
l'ultimo 1 si arriva in stato
di accettazione qualsiasi

Quindi L non può essere regolare.

Un DFA ha un numero finito di stati (diciamo n stati). Se l'automa deve leggere una stringa molto lunga (con più di n simboli), deve necessariamente passare più di una volta per lo stesso stato. Questo "ritorno" nello stesso stato crea un ciclo. Una volta individuato questo ciclo, l'automa può percorrerlo quante volte vuole (zero, una, cento...) e finirà comunque per accettare la stringa.

Proprietà di pompaggio dei DFA

- Il **pumping lemma** è basato su questa **idea del ciclo** e vale per **qualsiasi linguaggio regolare**.

il pumping lemma si basa

- In particolare, sulla seguente **proprietà di pompaggio** dei linguaggi riconosciuti da **DFA**

Dato un DFA A . Chiamo n il **numero degli stati** di A .

Osservazione: ogni stringa di lunghezza (almeno) n , data in input al DFA A , fa sì che l'automa **tocchi due volte uno stesso stato**.

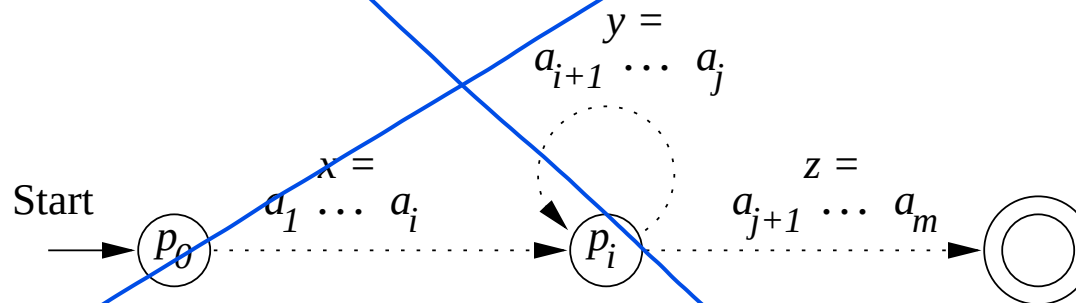
Dato un DFA A . Chiamo n il **numero degli stati** di A .

Osservazione: ogni stringa di lunghezza (almeno) n , data in input al DFA A , fa si' che l'automa **tocchi due volte uno stesso stato**.

Ogni $w = a_1 a_2 \dots a_m \in L(A) : |w| \geq n$ e', quindi, **scomponibile in tre stringhe** $w = xyz$, cioe'

- $x = a_1 a_2 \dots a_i$
- $y = a_{i+1} a_{i+2} \dots a_j$
- $z = a_{j+1} a_{j+2} \dots a_m$,

tali che il cammino di riconoscimento di w ha la seguente forma:



dove p_i e' lo stato raggiunto dopo l' i -esimo passo e $i < j \leq n$.

Poiché l'automa non può "distinguere" quante volte percorre il ciclo y , se accetta la stringa xyz , deve necessariamente accettare anche xz (zero giri), $xyyz$ (due giri), $xyyyz$, e così via. Se un linguaggio non permette questo "pompaggio" senza uscire dal linguaggio stesso, allora non può essere regolare.

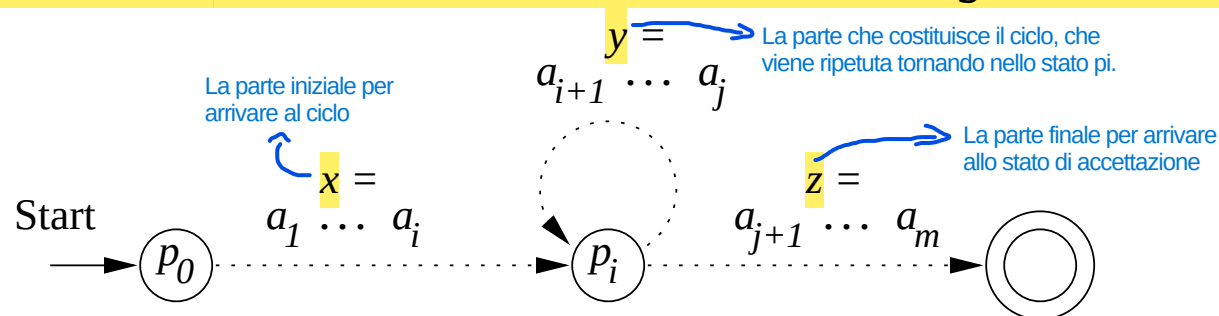
Dato un DFA A . Chiamo n il **numero degli stati** di A .

Osservazione: ogni stringa di lunghezza (almeno) n , data in input al DFA A , fa sì che l'automa **tocchi due volte uno stesso stato**.

Ogni $w = a_1a_2 \dots a_m \in L(A)$ ^{tale che} $|w| \geq n$ e', quindi, **scomponibile in tre stringhe** $w = xyz$, cioè

- $x = a_1a_2 \dots a_i$
- $y = a_{i+1}a_{i+2} \dots a_j$
- $z = a_{j+1}a_{j+2} \dots a_m$,

tali che il cammino di riconoscimento di w ha la seguente forma:



dove p_i e' lo stato raggiunto dopo l' i -esimo passo e $i < j \leq n$.

Quindi tutte le stringhe $xy^kz \in L(A)$, per ogni $k \geq 0$ (cioè, oltre a $w = xyz$, A accetta anche $xz, xyxz, xyxxz, \dots$)

Inoltre $y \neq \epsilon$ e $|xy| \leq n$ (poiché $i < j \leq n$).
 $k=0$ $k=1$ $k=2$ $k=3$

Questo teorema non serve a dimostrare che un linguaggio è regolare, ma serve quasi esclusivamente a dimostrare che un linguaggio NON è regolare. È una prova per assurdo.

Teorema 4.1.

Il Pumping Lemma per Linguaggi Regolari.

Sia L un linguaggio regolare.

la seguente condizione

Allora $\exists n \geq 1$ che soddisfa:

ogni $w \in L : |w| \geq n$ e' scomponibile in tre stringhe $w = xyz$ tali che

1. $y \neq \epsilon$ La parte centrale y (il ciclo) non può essere vuota. Devi "pompare" almeno un carattere.
2. $|xy| \leq n$ Il ciclo deve avvenire entro i primi n caratteri della stringa. Questo è fondamentale per le dimostrazioni perché limita dove può trovarsi la stringa y .
3. $\forall k \geq 0, xy^kz \in L$ Questa è la "proprietà di pompaggio". Se ripeti y per k volte, la nuova stringa deve appartenere ancora al linguaggio L .
Se $k=0$, abbiamo xz (abbiamo "saltato" il ciclo).
Se $k=2$, abbiamo $xyyz$ (abbiamo fatto il giro due volte).

Questa è la dimostrazione formale del perché il Pumping Lemma funziona.

↪ **Prova:** Supponiamo che L sia regolare.

Allora L è riconosciuto da un DFA A . Chiamo n il **numero degli stati** di A .

Sia $w = a_1a_2 \dots a_m \in L$, $m \geq n$.

p_i è lo stato in cui si trova l'automa dopo aver letto i primi i caratteri.

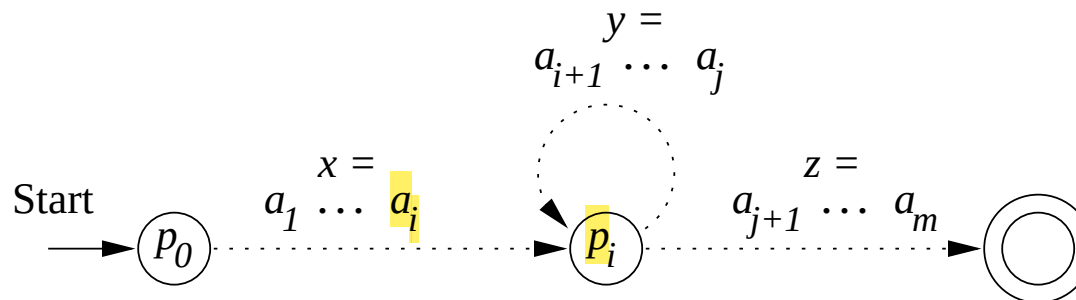
Sia $p_i = \hat{\delta}(q_0, a_1a_2 \dots a_i)$, dove q_0 è lo stato iniziale di A .

$\Rightarrow \exists i < j \leq n$ ^{tale che} $p_i = p_j$

Se leggiamo i primi n caratteri, tocchiamo $n+1$ stati. Dato che l'automa ha solo n stati distinti, è matematicamente impossibile che siano tutti diversi. Quindi, per forza, lo stato p_i deve essere uguale allo stato p_j per qualche indice tra 0 e n .

Ora $w = xyz$, dove

- $x = a_1 a_2 \dots a_i$ È la sequenza di simboli che porta l'automa dallo stato iniziale q_0 allo stato p_i .
- $y = a_{i+1} a_{i+2} \dots a_j$ È la sequenza che genera il ciclo, portando l'automa da p_i di nuovo in p_i (dato che $p_i = p_i$).
- $z = a_{j+1} a_{j+2} \dots a_m$ È la sequenza rimanente che porta l'automa dallo stato del ciclo p_i fino allo stato finale di accettazione.



Poiché y è un ciclo che parte e torna nello stesso stato (p_i), l'automa non può distinguere se quel ciclo è stato percorso una volta, dieci volte o nessuna. Se la stringa originale xyz portava a uno stato di accettazione, anche ogni versione "pompatà" ci porterà.

Quindi anche $xy^kz \in L$, per ogni $k \geq 0$.

Inoltre $y \neq \epsilon$ e $|xy| \leq n$ poiché $i < j \leq n$.

Poiché abbiamo stabilito che gli indici sono $i < j$, tra i e j deve esserci almeno un simbolo. Il ciclo non può essere "vuoto".

Abbiamo cercato lo stato ripetuto guardando solo i primi n caratteri della stringa. Poiché $j \leq n$, la lunghezza combinata di x (fino a i) e y (fino a j) non può superare il numero totale di stati n .

Per dire che L non é regolare, basta trovare una stringa che mostra che, tramite la proprietà di pompaggio, non faccia parte del linguaggio trovato un k qualsiasi

Esempio: sia L_{eq} il linguaggio delle stringhe con ugual numero di zeri e di uni.

Supponiamo che L_{eq} sia regolare. Allora, sia n dato dal pumping lemma, consideriamo $w = 0^n 1^n \in L$. (Per la dimostrazione, scegliere una stringa semplice)

Per il pumping lemma

$\exists x, y, z : w = xyz, |xy| \leq n, y \neq \epsilon$ e $xy^kz \in L_{eq}$, per ogni $k \geq 0$.

Essendo $|xy| \leq n$ si ha che y deve essere per forza fatta di soli 0, cioè:

$$w = \underbrace{0 \dots 0}_x \underbrace{0 \dots 0}_y \underbrace{0 \dots 0 111 \dots 111}_z$$

In particolare, quindi (per $k = 0$) dovrebbe valere $xz \in L_{eq}$, ma xz ha meno zeri di uni! (**assurdo**)

Quindi L_{eq} non é regolare

Se $k=0$, taglio y (cancello y)
Se $k=1$, mantengo la stringa com'è (mantengo y com'è)
Se $k \geq 2$, pompo y (aumento il contenuto di y)

Le proprietà di chiusura dei linguaggi regolari affermano che se applichi certe operazioni a linguaggi regolari, il risultato è sempre un altro linguaggio regolare. Questo è fondamentale perché ti permette di costruire linguaggi più complessi a partire da quelli più semplici

Proprieta' di chiusura dei linguaggi regolari

Siano L e M due linguaggi regolari. Allora i seguenti linguaggi sono regolari:

- ① • Unione: $L \cup M$
- ② • Intersezione: $L \cap M$
- ③ • Complemento: $\overline{L}$
- ④ • Differenza: $L \setminus M$
- ⑤ • Inversione: $L^R = \{w^R : w \in L\}$
- ⑥ • Chiusura: L^* . (Stella di Kleene)
- ⑦ • Concatenazione: $L.M$

Unione

①

Teorema 4.4. Per ogni coppia di linguaggi regolari L e M , $L \cup M$ e' regolare.

Prova. Sia $L = L(E)$ e $M = L(F)$. Allora $L(E + F) = L \cup M$ per definizione. (dove E ed F sono espressioni regolari)

Complementarietà

③

Teorema 4.5. Se L e' un linguaggio regolare su Σ , allora anche $\bar{L} = \Sigma^* \setminus L$ e' regolare.

Prova. Sia L riconosciuto da un DFA

DFA $A = (Q, \Sigma, \delta, q_0, F)$.

Sia $B = (Q, \Sigma, \delta, q_0, Q \setminus F)$. Allora $L(B) = \bar{L}$.

DFA

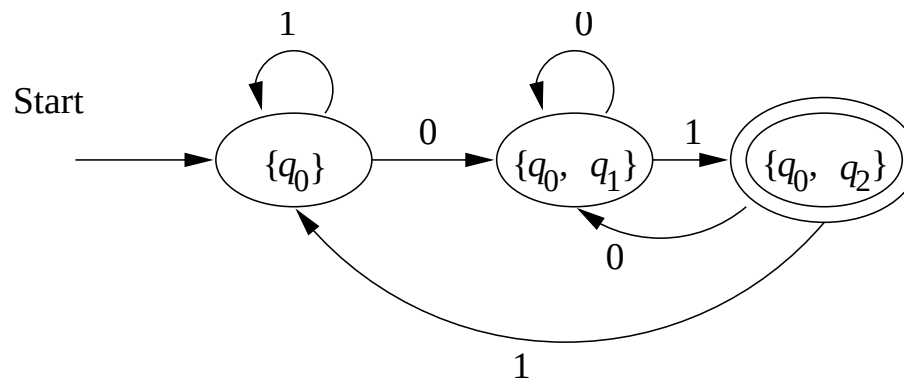
dove $\setminus$ é una differenza insiemistica

Prendo un DFA equivalente e lo dimostro invertendo gli stati di accettazione con quelli che non lo erano

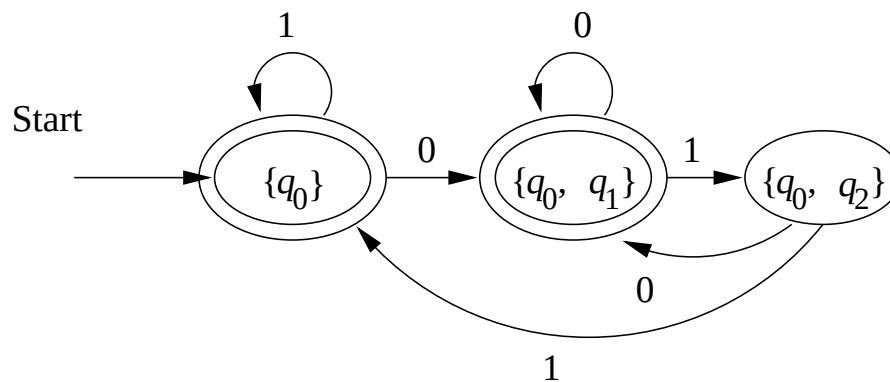


Esempio: Complementarietà

Sia L riconosciuto dal DFA qui sotto:



Allora $\bar{L}$ e' riconosciuto da:



Domanda: Espressione regolare per $\bar{L}$ a partire da una per L ?

Intersezione

Teorema 4.8. Se L e M sono regolari, allora anche $L \cap M$ e' regolare.

Prova. Per la legge di DeMorgan, $L \cap M = \overline{\overline{L} \cup \overline{M}}$, Sappiamo gia' che i linguaggi regolari sono chiusi rispetto al complemento e all'unione.

2A

Uso Unione e
Complemento per
definire
l'intersezione
come
operazione dei
linguaggi
regolari

La Simulazione Parallela si poteva usare anche per dimostrare l'Unione dove, per accettare, basta che un solo DFA dei due era in stato di accettazione

Intersezione (un'altra dimostrazione)

Teorema 4.8. Se L e M sono regolari, allora anche $L \cap M$ e' regolare.

Simulazione Parallela

Prova alternativa. Sia L il linguaggio di

DFA $A_L = (Q_L, \Sigma, \delta_L, q_L, F_L)$

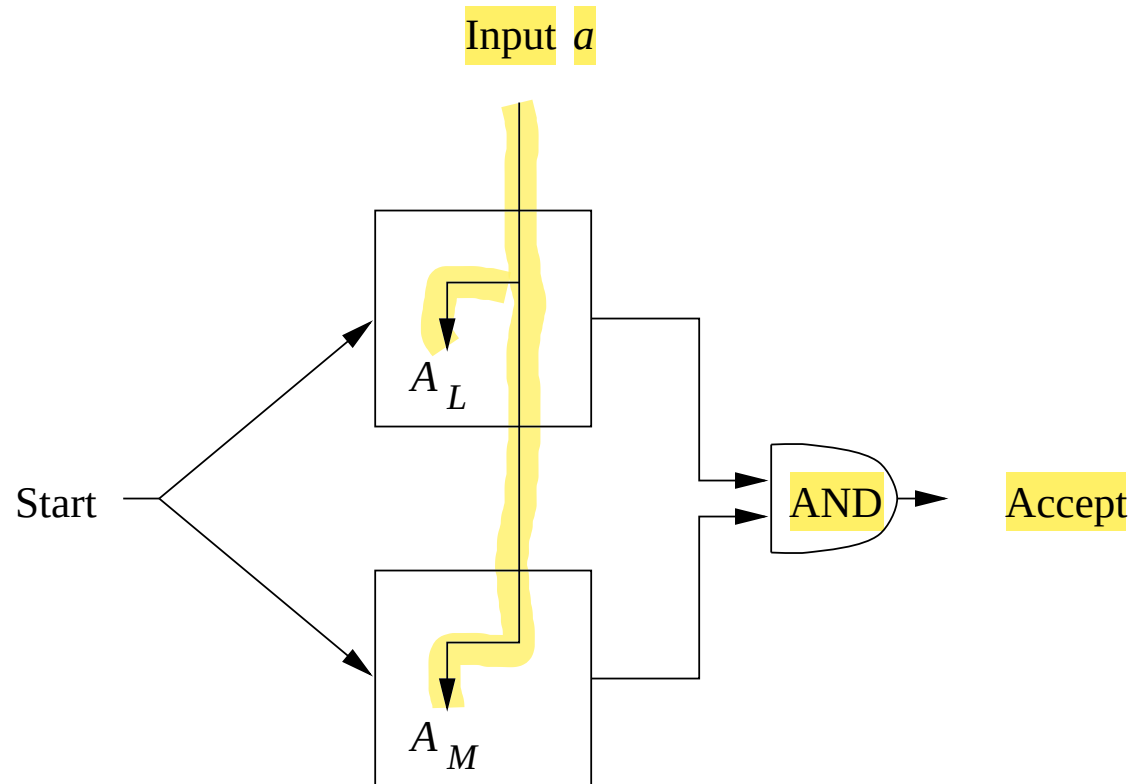
e M il linguaggio di

DFA $A_M = (Q_M, \Sigma, \delta_M, q_M, F_M)$

Assumiamo senza perdita di generalita' che entrambi gli automi siano deterministici.

Costruiremo un automa che simula A_L e A_M in parallelo, e accetta se e solo se sia A_L che A_M accettano. (cioè si é in intersezione quando entrambi sono in stato di accettazione)

Se A_L va dallo stato p allo stato s leggendo a , e A_M va dallo stato q allo stato t leggendo a , allora $A_{L \cap M}$ andrà dallo stato (p, q) allo stato (s, t) leggendo a . (entrambi leggono lo stesso input, ma vanno in stati diversi)



Formalmente

$$A_{L \cap M} = (\underbrace{Q_L \times Q_M}_{\text{prodotto cartesiano degli stati di AL e AM}}, \Sigma, \delta_{L \cap M}, \underbrace{(q_L, q_M)}_{\text{prodotto cartesiano degli stati di accettazione di AL e AM}}, F_L \times F_M),$$

dove

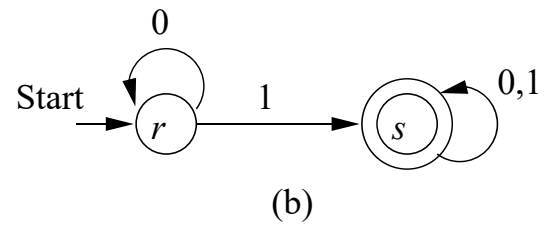
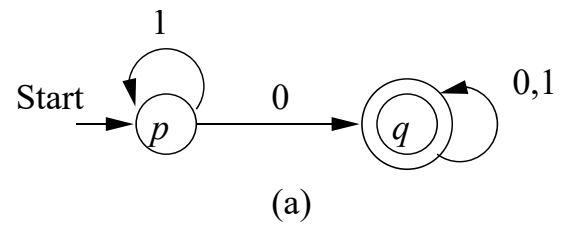
$$\delta_{L \cap M}(\underbrace{(p, q)}_{\text{stato in cui arrivo leggendo a per automa AL}}, \underbrace{a}_{\text{stato in cui arrivo leggendo a per automa AM}}) = (\delta_L(p, a), \delta_M(q, a))$$

Si può mostrare per induzione su $|w|$ che

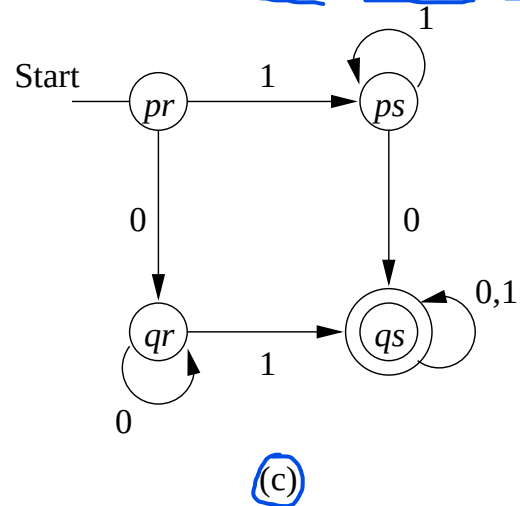
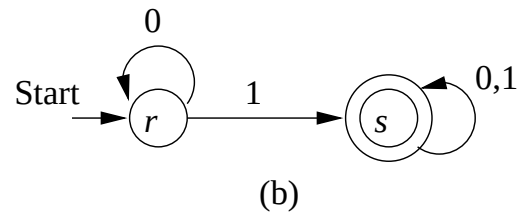
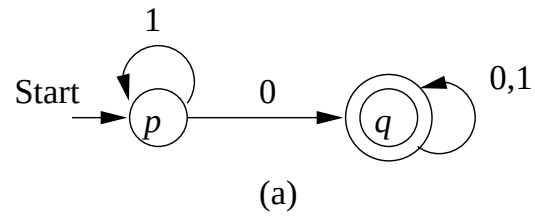
$$\hat{\delta}_{L \cap M}((q_L, q_M), w) = (\hat{\delta}_L(q_L, w), \hat{\delta}_M(q_M, w))$$

Se in uno stato di un automa, durante la simulazione parallela, non ha la transizione con il simbolo letto, allora in quel automa rimango in quello stato (non mi muovo in quell'automato: se l'altro ha transizioni con il simbolo letto, si può muovere al prossimo stato collegato)

Esempio: $(c) = (a) \times (b)$



Esempio: $(c) = (a) \times (b)$



④

Differenza

Teorema 4.10. Se L e M sono linguaggi regolari, allora anche $L \setminus M$ e' regolare.

Prova. Osserviamo che $L \setminus M = L \cap \overline{M}$. Sappiamo gia' che i linguaggi regolari sono chiusi rispetto al complemento e all'intersezione.

⑤

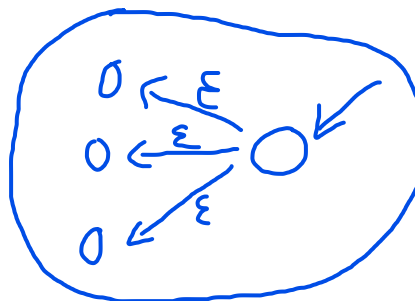
Inversione

Teorema 4.11. Se L è un linguaggio regolare, allora anche L^R (le stringhe w^R , cioè w lette al "reverse", con $w \in L$) è regolare.

Prova 1: Sia L riconosciuto da un FA A . Modifichiamo A per renderlo un FA per L^R :

1. Invertito Giriamo tutti gli archi. 
2. Rendiamo il vecchio stato iniziale l'unico stato finale.
3. Creiamo un nuovo stato iniziale p_0 , con $\delta(p_0, \epsilon) = F$ (i vecchi stati finali).

Poiché l'automa originale poteva avere più stati finali (F), non sapremmo da dove iniziare a leggere la stringa al contrario. Creiamo quindi un UNICO stato iniziale che, tramite delle epsilon-transizioni, ci porta istantaneamente in tutti i vecchi stati



Le proprietà di decisione dei linguaggi regolari sono quelle proprietà per le quali è possibile creare un algoritmo che stabilisca se una determinata caratteristica di un linguaggio si verifichi o meno. Tali proprietà sono "decidibili" perché un automa a stati finiti, o una macchina di Turing equivalente, può sempre fornire una risposta "sì" o "no" a queste domande, e la computazione terminerà sempre.

Proprieta' di decisione

- ①. $E' L = \emptyset?$ Problema del vuoto: Dato un linguaggio regolare L , si chiede se $L =$ insieme vuoto
- ②. $E' w \in L?$ Problema dell'appartenenza: Dato un linguaggio regolare L e una stringa w , si chiede se w appartiene ad L
- ③. Due descrizioni definiscono lo stesso linguaggio? Problema dell'Equivalenza: Dato un linguaggio regolare L , si chiede se le due descrizioni (che siano DFA, NFA o Espressioni Regolari) definiscono lo stesso linguaggio L

Due metodi per verificarlo

⑦ Testare se un linguaggio e' vuoto

1 $L(A) \neq \emptyset$ per FA A se e solo se uno stato finale e' raggiungibile dallo stato iniziale in A . Totale: $O(n^2)$ passi.

2 Oppure, possiamo guardare un'espressione regolare E di lunghezza s e vedere se $L(E) = \emptyset$ in $O(s)$ passi. Usiamo il seguente metodo:

La complessità è $O(s)$ perché leggiamo l'espressione una sola volta.

$E = F + G$ Allora $L(E)$ e' vuoto se e solo se sia $L(F)$ che $L(G)$ sono vuoti.

$E = F.G$ Allora $L(E)$ e' vuoto se e solo se $L(F)$ o $L(G)$ sono vuoti. (perché se uno vuoto, annulla anche l'altro con concatenazione)

$E = F^*$ Allora $L(E)$ non e' mai vuoto, perche' $\epsilon \in L(E)$.

$E = \epsilon$ Allora $L(E)$ non e' vuoto.

$E = a$ Allora $L(E)$ non e' vuoto.

$E = \emptyset$ Allora $L(E)$ e' vuoto.

②

Controllare l'appartenenza

Per controllare se $w \in L(A)$ per DFA A , simuliamo A su w . Se $|w| = n$, questo prende $O(n)$ passi.

Se A è un NFA e ha s stati, simulare A su w prende $O(ns^2)$ passi. Lo stesso per ϵ -NFA calcolando preventivamente le ECLOSE di tutti gli stati.

Se $L = L(E)$, per l'espressione regolare E di lunghezza s , prima convertiamo E in un ϵ -NFA con $2s$ stati. Poi simuliamo w su questo automa, in $O(ns^2)$ passi.

Esploro tutti i possibili stati per ogni simbolo di w (dove $|w|=n$)

Ad ogni passo, nel caso peggiore, potresti essere in tutti gli s stati. Per ogni stato, devi controllare tutte le possibili transizioni verso gli altri stati (altri s stati). Da qui deriva il fattore s^2 per ogni carattere letto di w .

Due stati di un DFA sono distinguibili, quando, con uno dei due stati, non si arriva in stato di accettazione con stessa stringa (la stringa può essere anche vuota, quindi epsilon)

Due stati di un DFA sono equivalenti, se si arriva a stati di accettazione con stessa stringa da due stati diversi del DFA

Equivalenza e Minimizzazione di Automi

Sia $A = (Q, \Sigma, \delta, q_0, F)$ un DFA, e $\{p, q\} \subseteq Q$. Definiamo

$$p \equiv q \Leftrightarrow \forall w \in \Sigma^* : \hat{\delta}(p, w) \in F \text{ se e solo se } \hat{\delta}(q, w) \in F$$

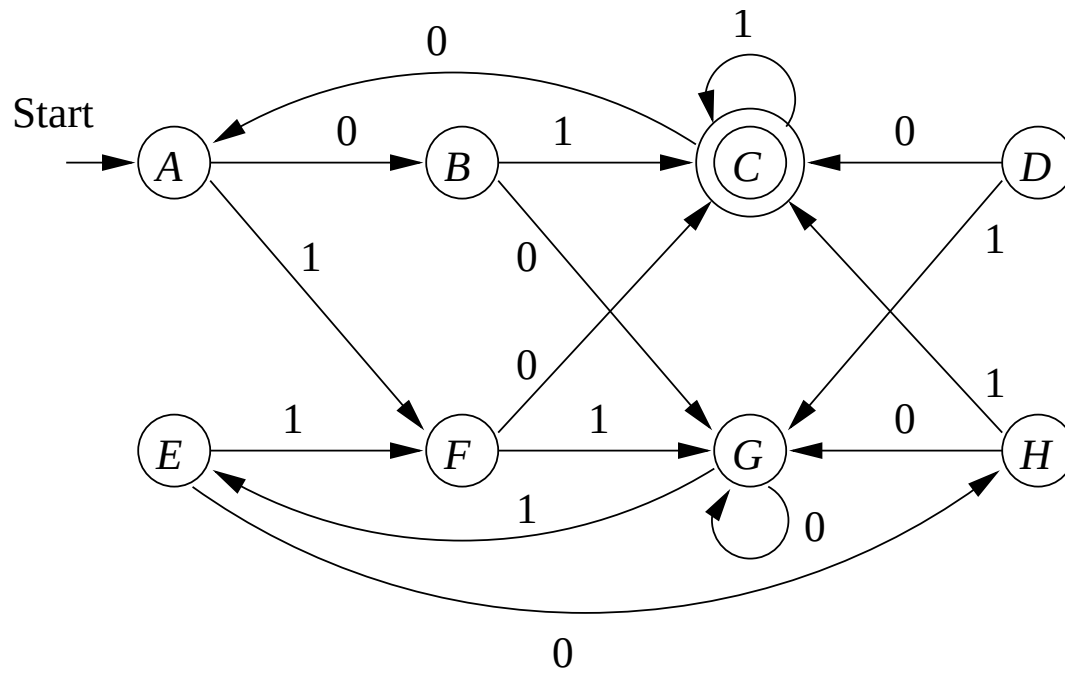
- Se $p \equiv q$ diciamo che p e q sono *equivalenti* (per ogni w)
- Se $p \not\equiv q$ diciamo che p e q sono *distinguibili* (se trovo anche solo una stringa che mi dice che non sono equivalenti)

In altre parole: p e q sono distinguibili se e solo se

$$\exists w : \hat{\delta}(p, w) \in F \text{ e } \hat{\delta}(q, w) \notin F, \text{ o viceversa}$$

Per dire che due stati sono equivalenti, devo controllare infinite stringhe. Per dire che due stati sono distinguibili, mi basta trovare una stringa che mi da conferma di ciò. Se due stati, tramite una stringa o una parte di essa, arrivano allo stesso stato, un sottoinsieme di stringhe rende quei due stati indistinguibili, dato che dopo lo stesso stato raggiunto avranno stesso percorso per arrivare allo stato di accettazione

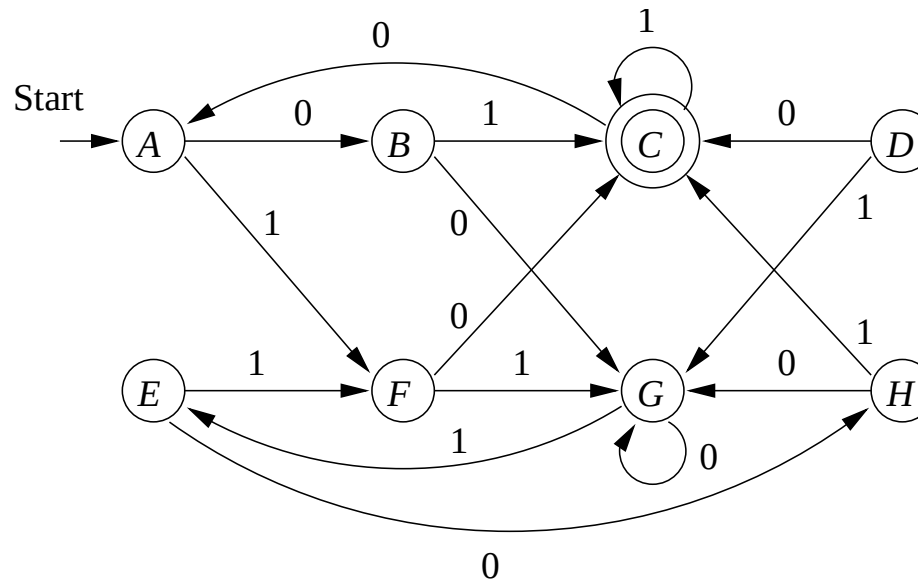
Esempio:



$$\hat{\delta}(C, \epsilon) \in F, \hat{\delta}(G, \epsilon) \notin F \Rightarrow C \neq G$$

$$\hat{\delta}(A, 01) = C \in F, \hat{\delta}(G, 01) = E \notin F \Rightarrow A \neq G$$

Cosa si puo' dire su A e E ?



$$\hat{\delta}(A, \epsilon) = A \notin F, \hat{\delta}(E, \epsilon) = E \notin F$$

$$\hat{\delta}(A, 1) = F = \hat{\delta}(E, 1)$$

$$\text{Quindi } \hat{\delta}(A, 1x) = \hat{\delta}(E, 1x) = \hat{\delta}(F, x)$$

$$\hat{\delta}(A, 0) = B \notin F \quad \hat{\delta}(E, 0) = H \notin F$$

$$\hat{\delta}(A, 00) = G = \hat{\delta}(E, 00) \quad \hat{\delta}(A, 01) = C = \hat{\delta}(E, 01)$$

Conclusione: $A \equiv E$

Possiamo calcolare coppie di stati distinguibili con il seguente metodo induttivo (*algoritmo riempi-tabella*):

Base: Se $p \in F$ e $q \notin F$, allora $p \not\equiv q$.

Caso Base (metto x alle coppie distinguibili, a causa del fatto che sono di diverso tipo (coppia finale-nonfinale))

Induzione: Se $\exists a \in \Sigma : \delta(p, a) \not\equiv \delta(q, a)$, allora $p \not\equiv q$.

Per ogni passata di induzione, controllo sulla matrice se, per ogni coppia, ho altre coppie distinguibili (applicando le trasformazioni per ogni simbolo dell'alfabeto sigma, dalla coppia analizzata, controllo la nuova coppia in cui arrivo e verifico se ha la X sulla tabella)

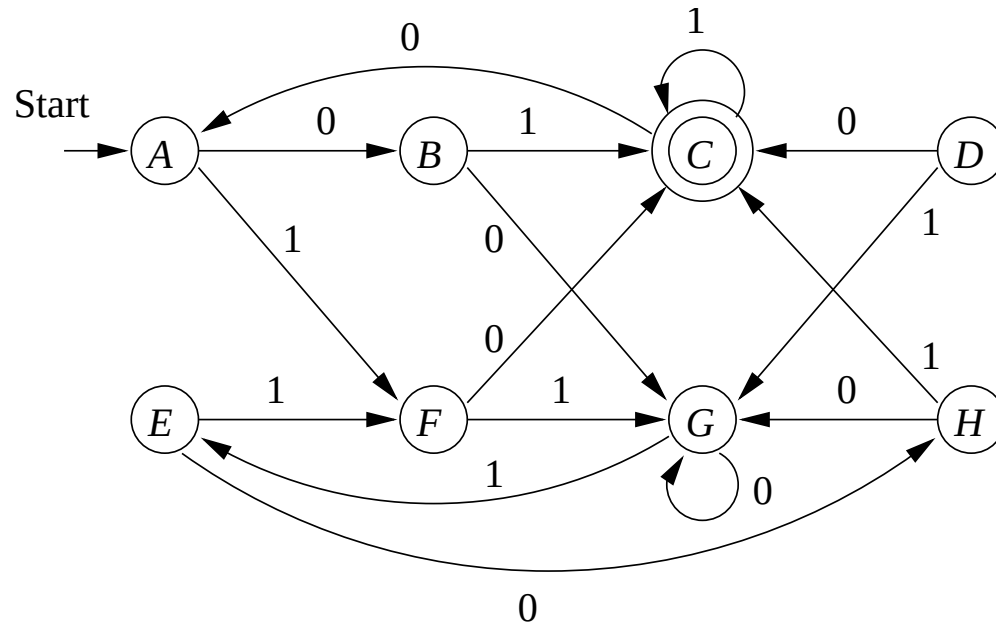
Esempio: Applichiamo l'algoritmo ad A:

B	x						
C	x	x					
D	x	x	x				
E		x	x	x			
F	x	x	x		x		
G	x	x	x	x	x	x	
H	x		x	x	x	x	x
	A	B	C	D	E	F	G

Ho solo la matrice triangolare inferiore (quindi senza diagonale e parte superiore, perché é simmetrica e non mi interessa confrontare un stato con se stesso)

Esempio: sigma={0,1}
 (A,B) con 0 vado in (B,G) e non posso dire niente perché la loro casella é vuota (quindi ancora considerate equivalenti)
 (A,B) con 1 vado in (C,F) e (C,F) é distinguibile, quindi anche (A,B) é distinguibile

Se nell'ultima passata ho definito almeno una nuova coppia distinguibile, devo applicare un'altra passata induttiva. Termino quando non ho trovato nuove coppie distinguibili



<i>B</i>							
<i>C</i>							
<i>D</i>							
<i>E</i>							
<i>F</i>							
<i>G</i>							
<i>H</i>							
	<i>A</i>	<i>B</i>	<i>C</i>	<i>D</i>	<i>E</i>	<i>F</i>	<i>G</i>

Teorema 4.20: Se p e q non sono distinti dall'algoritmo, allora $p \equiv q$.

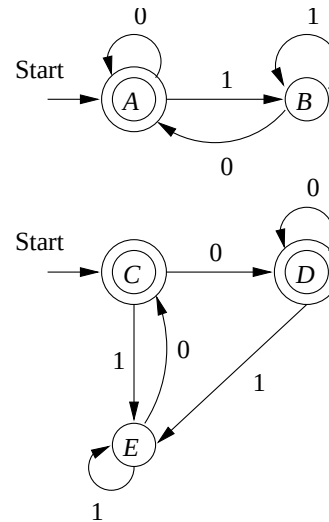
Testare l'equivalenza di linguaggi regolari

Siano L e M linguaggi regolari (descritti in qualche forma).

Per testare se $L = M$

1. convertiamo sia L che M in DFA.
2. Immaginiamo il DFA che e' l'unione dei due DFA (non importa se ha due stati iniziali) ed applico l'algoritmo del riempi-tabella come un unico DFA
3. Se l'algoritmo dice che i due stati iniziali sono distinguibili, allora $L \neq M$, altrimenti $L = M$.

Esempio:



Il risultato dell'algoritmo e'

<i>B</i>	x			
<i>C</i>	x	x		
<i>D</i>		x		
<i>E</i>	x		x	x
	<i>A</i>	<i>B</i>	<i>C</i>	<i>D</i>

Quindi i due automi sono equivalenti.

Dopo aver trovato gli stati equivalenti, fondiamo insieme gli stati equivalenti, creando le classi di equivalenza, per minimizzare l'automa (Dalle classi, creo degli stati con gli stati di quella classe). Le classi vengono create in base alle relazioni di equivalenza (le classi sono insiemi disgiunti e la loro unione dà l'insieme di tutti gli elementi, nel nostro caso gli stati dell'automa. Gli elementi/stati, senza relazioni di equivalenza, creano una classe da soli)

Minimizzazione di DFA

Algoritmo per minimizzare un DFA:

mettiamo insieme tutti gli stati equivalenti (considerando come stati le classi di equivalenza $p/\equiv$ al posto di p)
ed eliminiamo eventuali stati non raggiungibili dallo stato iniziale.

Esempio: Il DFA di prima ha le seguenti classi di equivalenza:
 $\{\{A, E\}, \{B, H\}, \{C\}, \{D, F\}, \{G\}\}$.

Il DFA unione di prima ha le seguenti classi di equivalenza:
 $\{\{A, C, D\}, \{B, E\}\}$.

Nota: $p/\equiv$ è classe di equivalenza della relazione di equivalenza $\equiv$ (cioè una relazione riflessiva, simmetrica, e transitiva).

Proprietà delle Classi di Equivalenza:

- Riflessiva: P equivalente P
- Simmetrica: A equivalente B e B equivalente A
- Transitiva: A equivalente B e A equivalente C , allora B equivalente C

L'idea è che, una volta raggruppati gli stati equivalenti tra loro in "famiglie" (chiamate classi di equivalenza), non ci serve più considerare ogni singolo stato, ma solo la classe a cui appartiene.

Quindi, **per minimizzare un DFA** $A = (Q, \Sigma, \delta, q_0, F)$, dove assumiamo di aver già eliminato tutti gli stati non raggiungibili dallo stato iniziale, costruiamo un DFA $B = (Q/\equiv, \Sigma, \gamma, q_0/\equiv, F/\equiv)$, dove

In parole povere: per sapere dove andare dalla classe dove ho p leggendo il simbolo a , guarda dove andava lo stato p nell'automata originale e prendi la classe di arrivo.

$$\gamma(p/\equiv, a) = \delta(p, a)/\equiv$$

e $Q/\equiv$ è l'insieme delle classi di equivalenza degli stati di Q ($F/\equiv$ l'insieme di quelle degli stati accettanti).

Il nuovo stato iniziale è la classe che contiene il vecchio stato iniziale q_0 .

classe che ha solo stati accettanti

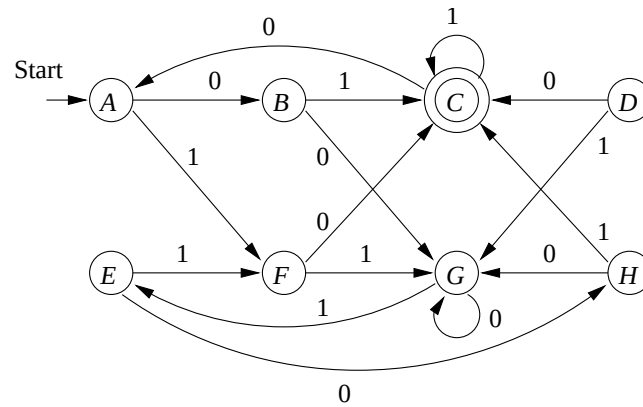
Affinché B sia ben definito, dobbiamo mostrare che

$$\text{Se } p \equiv q \text{ allora } \delta(p, a) \equiv \delta(q, a)$$

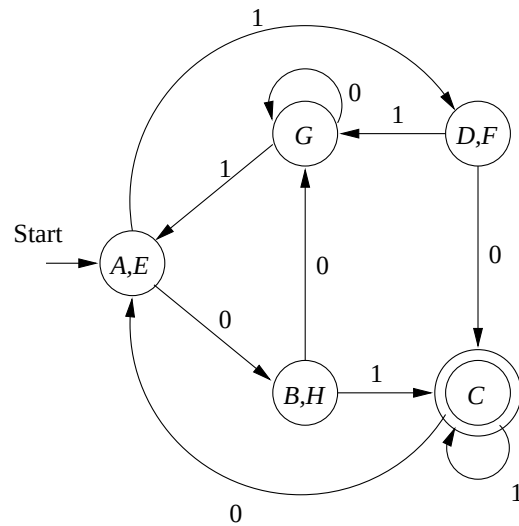
Se $\delta(p, a) \not\equiv \delta(q, a)$, allora l'algoritmo concluderebbe $p \not\equiv q$, quindi B è ben definito.

Per far sì che l'automata minimo funzioni, la scelta dello stato all'interno della classe non deve influenzare il risultato. Il requisito: Se p e q sono nello stesso gruppo (p e q equivalenti), allora dopo aver letto lo stesso carattere a , devono finire ancora in stati che appartengono allo stesso gruppo ($\delta(p, a)$ e $\delta(q, a)$ equivalenti). La coerenza: Se così non fosse (ovvero se leggendo a si finisse in classi diverse), l'algoritmo di minimizzazione avrebbe dovuto capire che p e q non erano davvero equivalenti fin dall'inizio e li avrebbe separati.

Esempio: Possiamo minimizzare

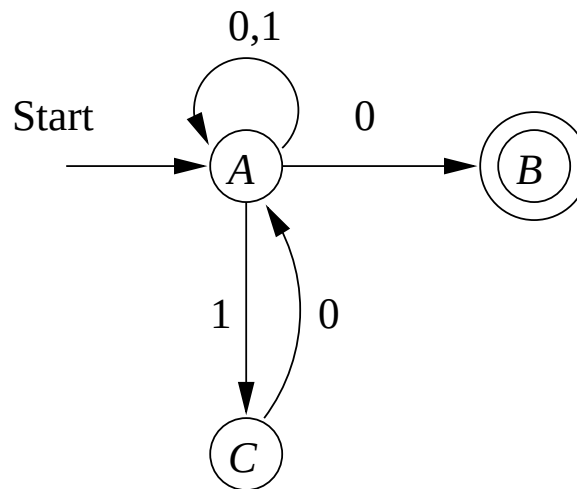


per ottenere



Notare: Non possiamo applicare l'algoritmo a NFA.

Per esempio, per minimizzare



dovrei poter rimuovere lo stato C . (perché, rimuovendo C , avrei comunque un Automa equivalente)

Ma $A \neq C$.